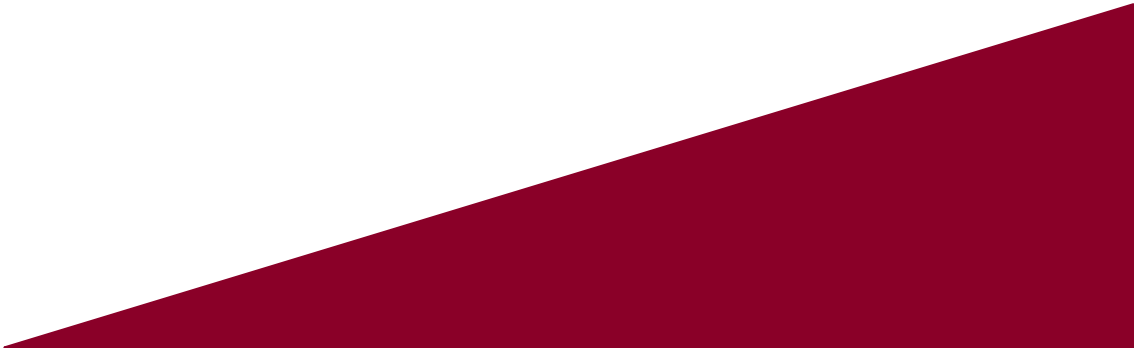




인공지능

Artificial Intelligence



Computer Vision

- 시각 인지 기술의 큰 흐름

- CNN 아키텍처 기반 연구 (2012~) ← 6장에서 배웠음. 한번 더?
 - 신경망 연구의 시작, 다양한 CNN 백본 모델을 제안하며 성능 향상을 가져옴
- Vision Transformer 아키텍처 기반 연구 (2021~)
 - 자연어 분야에서 발표된 Transformer (2017) 아키텍처를 비전 분야 아키텍처로 도입한 사례
 - 다양한 ViT 백본 모델을 제안하며 성능 향상을 가져옴
- VLM (Vision Language Model) 기반 연구 (2021~)
 - 기존 비전 모델은 수천~수만 개 클래스에 대해 고정된 Label만 탐지 가능 (Close-set)
 - VLM 모델은 텍스트 임베딩 기반으로 작동하기 때문에, 사전에 본 적 없는 클래스도 언어로 설명하면 탐지 가능 (Zero-shot 가능)

Computer Vision

- 시각 인지 기술의 큰 흐름

- CNN 아키텍처 기반 연구 (2012~)

- 성능이 고도화되는 CNN 백본 모델 이 제안됨

- CNN 백본의 발달은 feature representation 능력의 향상을 가져와, 다른 비전 테스트를 위한 사전 학습 모델로 사용 가능

- 백본의 발전은 깊이(Depth), 효율(Efficiency), 모듈화(Modularity)로 진화함

```
LeNet (1998)
↓
AlexNet (2012)
↓
VGG (2014)
↓
GoogLeNet → Inception 시리즈 (2014~2016)
↓
ResNet (2015) → DenseNet / ResNeXt (2017)
↓
MobileNet / Xception / SEnet (2017~2018)
↓
EfficientNet (2019)
↓
ConvNeXt (2022)
```

AlexNet : ImageNet 우승, CNN 르네상스 시작

VGG: 단순 규칙적 구조(모듈화), 추후 백본의 표준

GoogLeNet: 파라미터 효율적, 1x1 conv 사용

Inception V2/V3: 계산 효율, 수렴안정성

Inception V4: 깊은 inception의 완성형

DenseNet : 파라미터 절감, Feature reuse 극대화

MobileNet: 경량 CNN 대표, 모바일용 최적화

EfficientNet: 파라미터 효율 최고 수준

ConvNeXt : ViT 스타일로 재해석한 CNN

Computer Vision

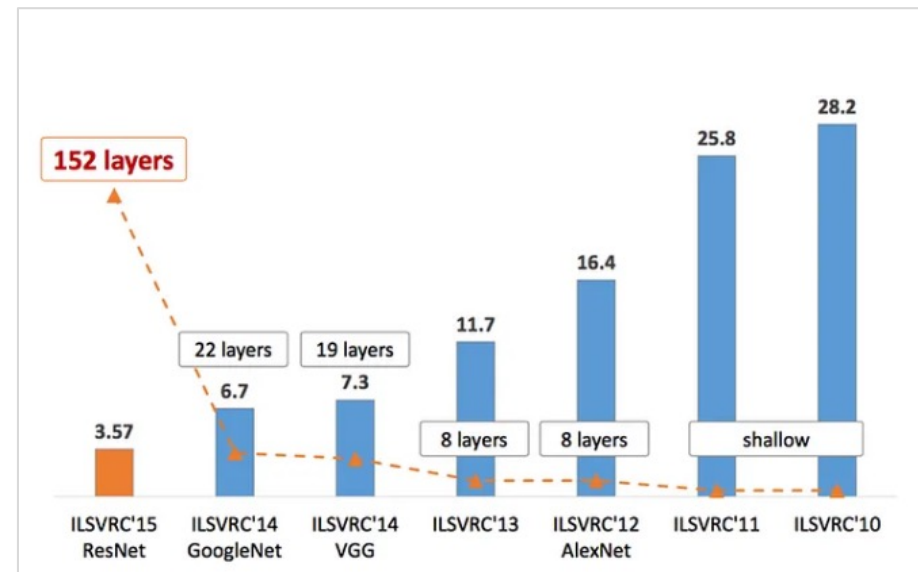
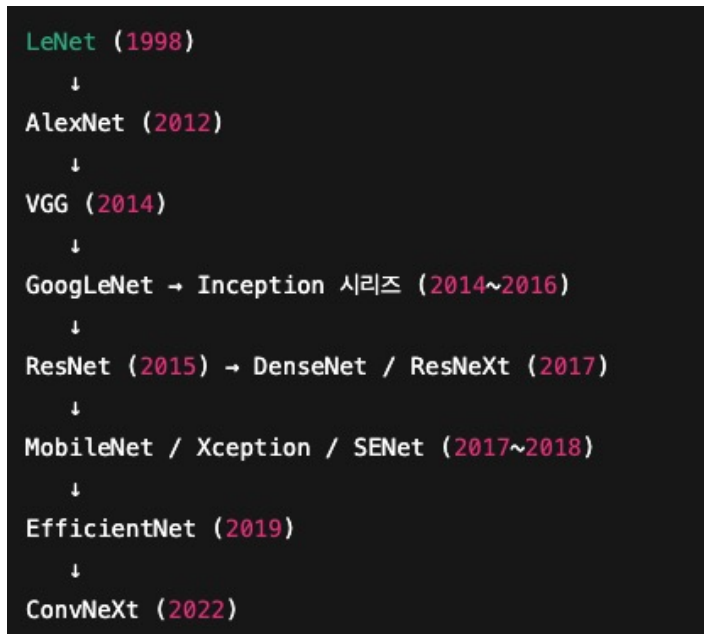
- 시각 인지 기술의 큰 흐름

- CNN 아키텍처 기반 연구 (2012~)

- 성능이 고도화되는 CNN 백본 모델 이 제안됨

- CNN 백본의 발달은 feature representation 능력의 향상을 가져와, 다른 비전 테스트를 위한 사전 학습 모델로 사용 가능

- 백본의 발전은 깊이(Depth), 효율(Efficiency), 모듈화(Modularity)로 진화함



Top5-error 가 줄어들 수록 (세로바), 네트워크는 깊어짐(주황색 점선)

Computer Vision

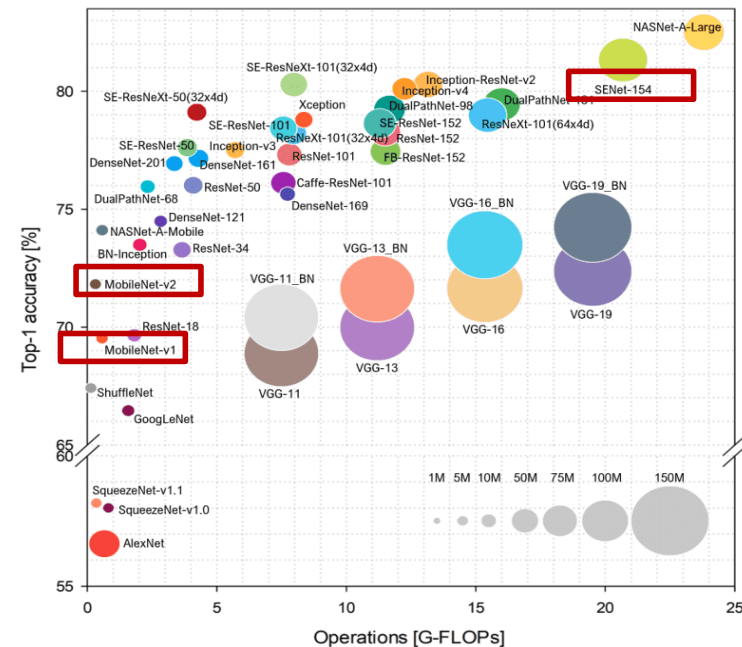
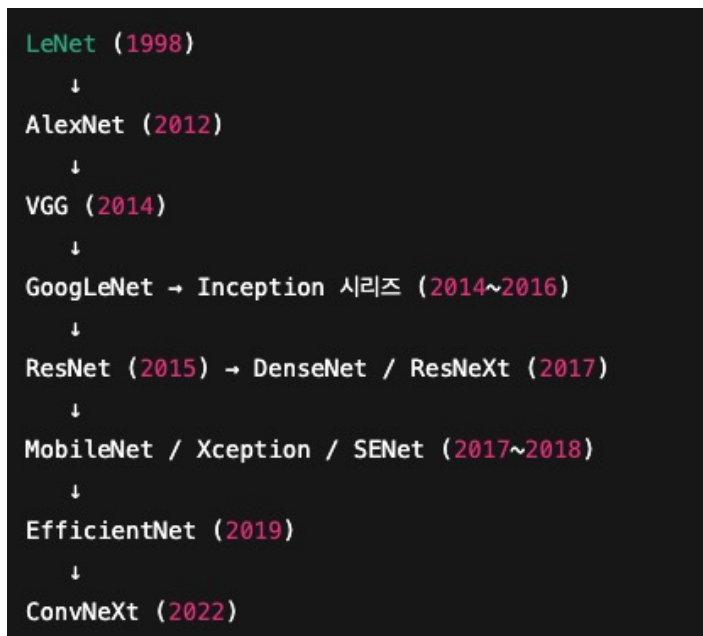
- 시각 인지 기술의 큰 흐름

- CNN 아키텍처 기반 연구 (2012~)

- 성능이 고도화되는 CNN 백본 모델 이 제안됨

- CNN 백본의 발달은 feature representation 능력의 향상을 가져와, 다른 비전 테스트를 위한 사전 학습 모델로 사용 가능

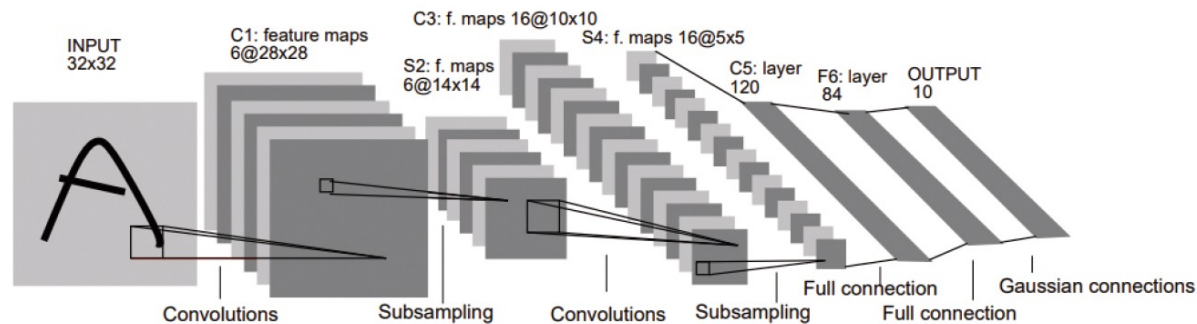
- 백본의 발전은 깊이(Depth), 효율(Efficiency), 모듈화(Modularity)로 진화함



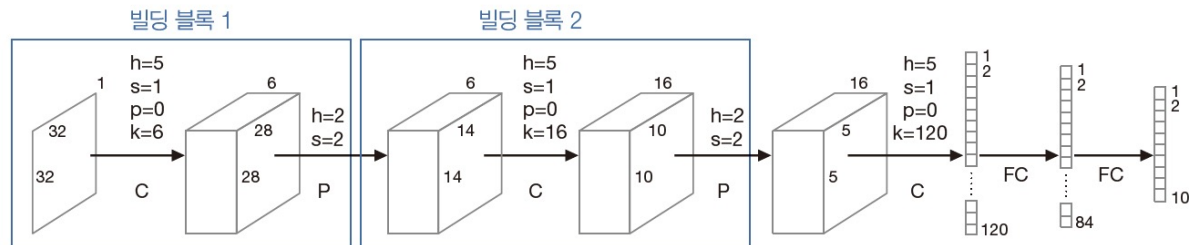
Benchmark Analysis of Representative Deep Neural Network Architectures, 2018

Computer Vision

- 시각 인지 기술의 큰 흐름
 - CNN 아키텍처 기반 연구 (2012~)
 - LeNet-5 (1998)
 - 성공적인 첫 CNN 아키텍처, 필기 인식 성공



(a) [LeCun1998]의 그림



(b) 이 책의 방식에 따른 그림

그림 6-11 LeNet-5의 구조(h는 커널의 크기, s는 보폭, p는 덧대기, k는 커널 개수)

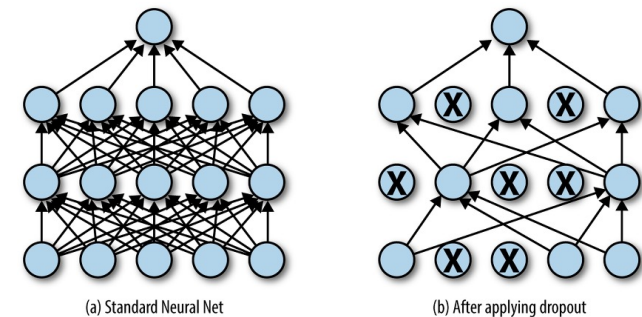
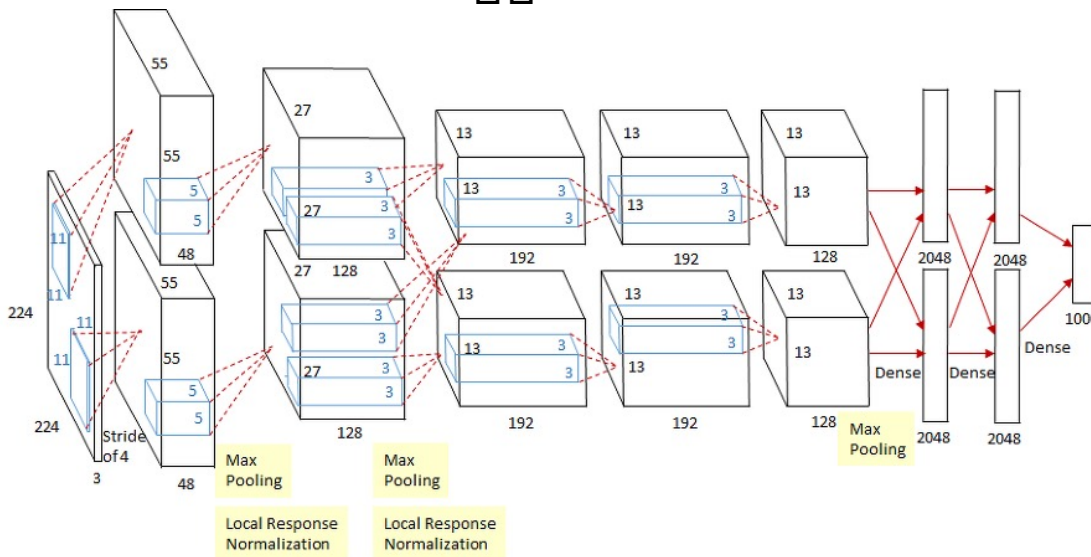
Computer Vision

■ 시각 인지 기술의 큰 흐름

■ CNN 아키텍처 기반 연구 (2012~)

■ AlexNet (2012)

- 6천만 개의 매개변수가 존재하며, 2개의 GTX 580 3GB GPU에서 학습하는데 5~6일 소요됨
- AlexNet은 ReLU 비선형성을 사용하여 빠르게 훈련할 수 있음을 보여줌(Tanh의 5배 빠름)
- 과적합 방지를 위해 Data Augmentation(Mirroring, Random Crops)과 Dropout을 도입함



Dropout으로 삭제된 뉴런은 학습과정에서
정방향 또는 역방향 전파에 기여하지 않음

```
LeNet (1998)
↓
AlexNet (2012)
↓
VGG (2014)
↓
GoogLeNet → Inception 시리즈 (2014~2016)
↓
ResNet (2015) → DenseNet / ResNeXt (2017)
↓
MobileNet / Xception / SNet (2017~2018)
↓
EfficientNet (2019)
↓
ConvNeXt (2022)
```


Computer Vision

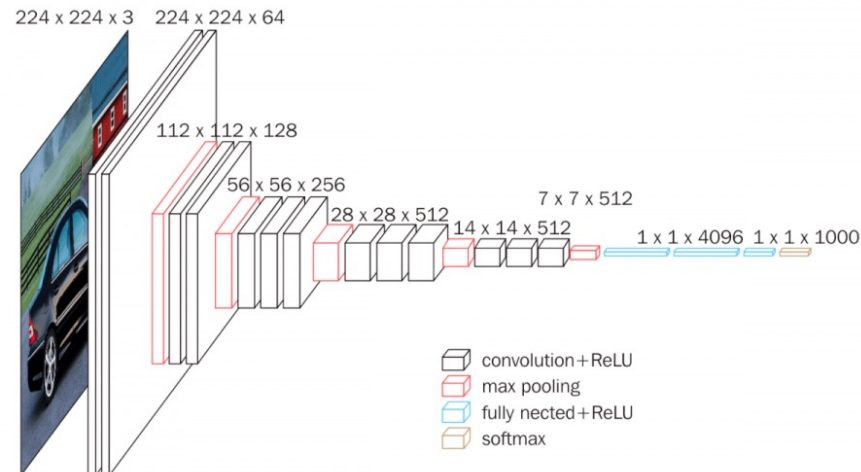
- 시각 인지 기술의 큰 흐름

- CNN 아키텍처 기반 연구 (2012~)

- VGG-16, VGG-19 (2014)

- VGG는 Visual Geometry Group의 약자, VGG 뒤의 숫자는 컨볼루션 레이어의 수를 의미함
 - VGG-16: 13개의 CNN 층 + 3개의 FC 층
 - 큰 커널 크기의 필터를 여러 3x3 커널 크기 필터로 교체하여 AlexNet에 비해 성능 개선 성공
 - 단순하고 반복적인 획일화된 구조가 매력적임
 - VGG-16의 매개변수는 1억 3800만개로 Titan Black 6GB GPU를 사용하여 몇 주간 학습

```
LeNet (1998)
↓
AlexNet (2012)
↓
VGG (2014)
↓
GoogLeNet → Inception 시리즈 (2014~2016)
↓
ResNet (2015) → DenseNet / ResNeXt (2017)
↓
MobileNet / Xception / SNet (2017~2018)
↓
EfficientNet (2019)
↓
ConvNeXt (2022)
```



Computer Vision

■ 시각 인지 기술의 큰 흐름

- CNN 아키텍처 기반 연구 (2012~)

- VGG-16, VGG-19 (2014)

- 입력은 고정된 크기 필요 (224x224), 학습데이터의 평균 값을 빼는 부분 필수!

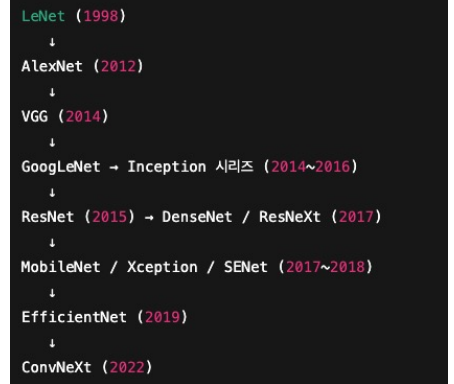


Table 1: **ConvNet configurations** (shown in columns). The depth of the configurations increases from the left (A) to the right (E), as more layers are added (the added layers are shown in bold). The convolutional layer parameters are denoted as “conv(receptive field size)-(number of channels)”. The ReLU activation function is not shown for brevity.

ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224 × 224 RGB image)					
conv3-64	conv3-64 LRN	conv3-64 conv3-64	conv3-64	conv3-64	conv3-64
max pool					
conv3-128	conv3-128	conv3-128 conv3-128	conv3-128	conv3-128	conv3-128
max pool					
conv3-256	conv3-256	conv3-256	conv3-256	conv3-256 conv3-256	conv3-256
max pool					
conv3-512	conv3-512	conv3-512	conv3-512	conv3-512 conv3-512	conv3-512
max pool					
conv3-512	conv3-512	conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512
max pool					
FC-4096					
FC-4096					
FC-1000					
softmax					

Table 2: **Number of parameters** (in millions).

Network	A, A-LRN	B	C	D	E
Number of parameters	133	133	134	138	144

Computer Vision

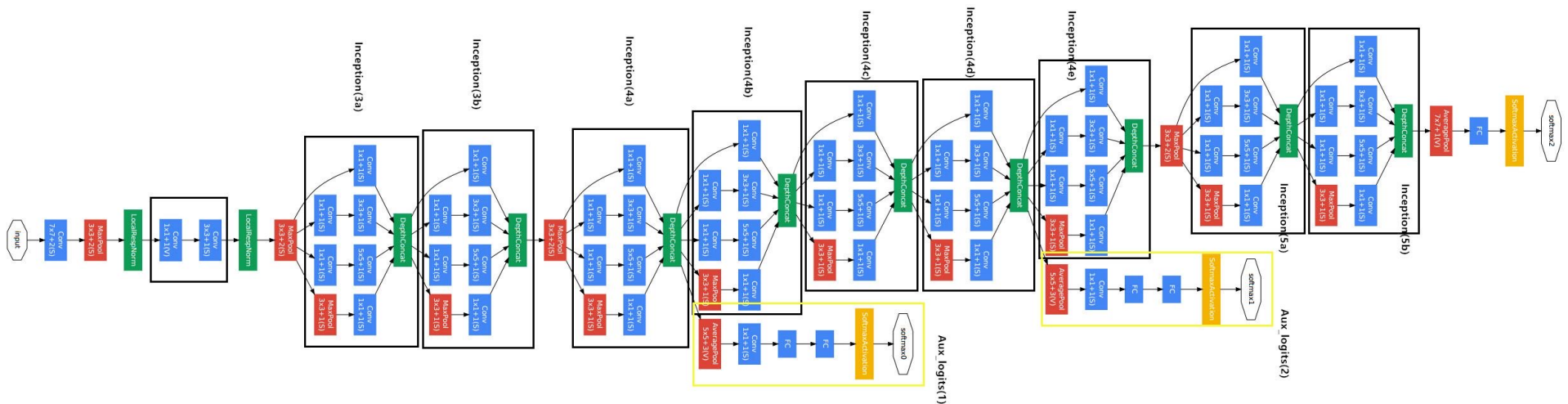
■ 시각 인지 기술의 큰 흐름

■ CNN 아키텍처 기반 연구 (2012~)

■ Inception (2014)

- 속도와 정확성을 고려하여 여러 버전이 탄생 (V1, V2, V3, -ResNet)
- 깊게만 쌓지 말고, 깊이와 너비 모두 깊게 쌓아보자
- 층이 깊어 발생하는 그레디언트 소실 문제는 보조 분류기(Auxiliary classifier)로 해결
- 복잡한 컨볼루션 연산을 decompose하여 효율적인 연산법 발견 (다음 슬라이드에서..)

LeNet (1998)
↓
AlexNet (2012)
↓
VGG (2014)
↓
GoogLeNet → Inception 시리즈 (2014~2016)
↓
ResNet (2015) → DenseNet / ResNeXt (2017)
↓
MobileNet / Xception / SNet (2017~2018)
↓
EfficientNet (2019)
↓
ConvNeXt (2022)



Computer Vision

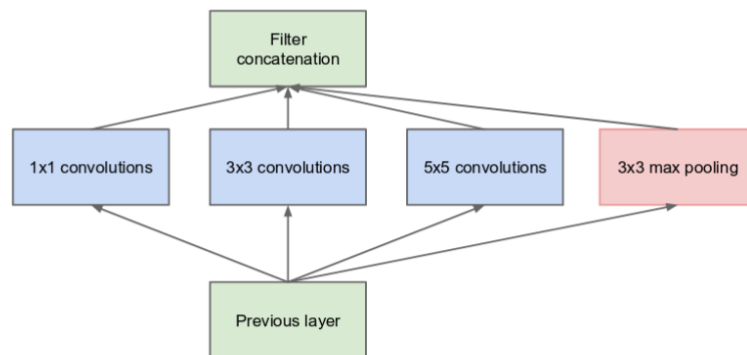
- 시각 인지 기술의 큰 흐름

- CNN 아키텍처 기반 연구 (2012~)

- Inception-V1 (2014), GoogLeNet

- 네트워크를 깊게 하면 표현력이 좋아지지만 연산량이 많아지고, 얇게 쌓아 연산량을 줄이면 표현력이 떨어지는 문제가 발생. → 희소성(Sparsity)을 통해 해당 문제를 해결 하려함
 - 희소성이란 CNN의 부분연결성과 유사한 개념으로, 연결이 적지만 효율적인 구조를 의미함
 - 그림(a)와 같이 병렬 Conv를 통해 sparse 연결을 dense 연산으로 근사
 - 이런 병렬 Conv 구조는 연산량 대비 표현력 향상, 다양한 공간적 특징 학습 가능함을 보였음

```
LeNet (1998)
↓
AlexNet (2012)
↓
VGG (2014)
↓
GoogLeNet → Inception 시리즈 (2014~2016)
↓
ResNet (2015) → DenseNet / ResNeXt (2017)
↓
MobileNet / Xception / SNet (2017~2018)
↓
EfficientNet (2019)
↓
ConvNeXt (2022)
```



(a) Inception module, naïve version

입력 feature map (filter concatenation 출력)의 일부는 작은 receptive field (1×1)에, 일부는 큰 receptive field (5×5)에 반응하도록 되어 전체 네트워크가 “희소하게 활성화” 되는 효과를 가져옴

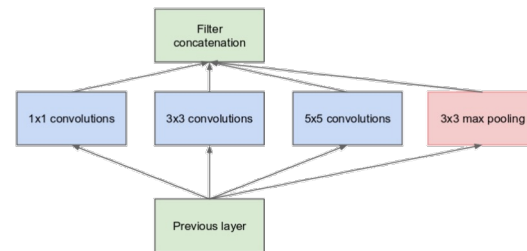
Computer Vision

시각 인지 기술의 큰 흐름

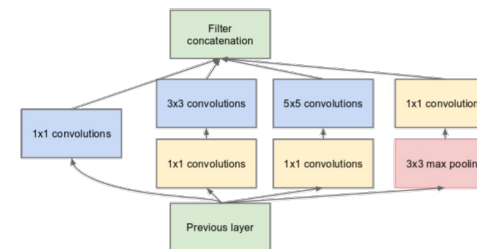
- CNN 아키텍처 기반 연구 (2012~)

Inception V1 (2014)

- 값비싼 3x3 및 5x5 convolution 전에 1x1 convolution을 사용하여 차원 축소



(a) Inception module, naïve version



(b) Inception module with dimension reductions

Inception V2

- 5x5 컨볼루션을 2개의 3x3 컨볼루션 연산으로 인수분해하여 계산 속도 향상
- 3x3 컨볼루션은 3x1다음에 1x3 을 통해 동일한 표현 효과 가능

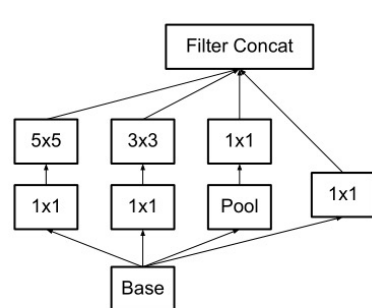
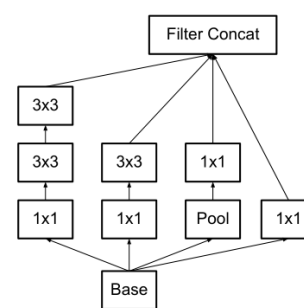


Figure 4. Original Inception module as described in [20].

==>



==>

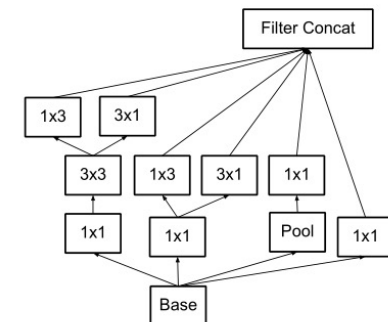


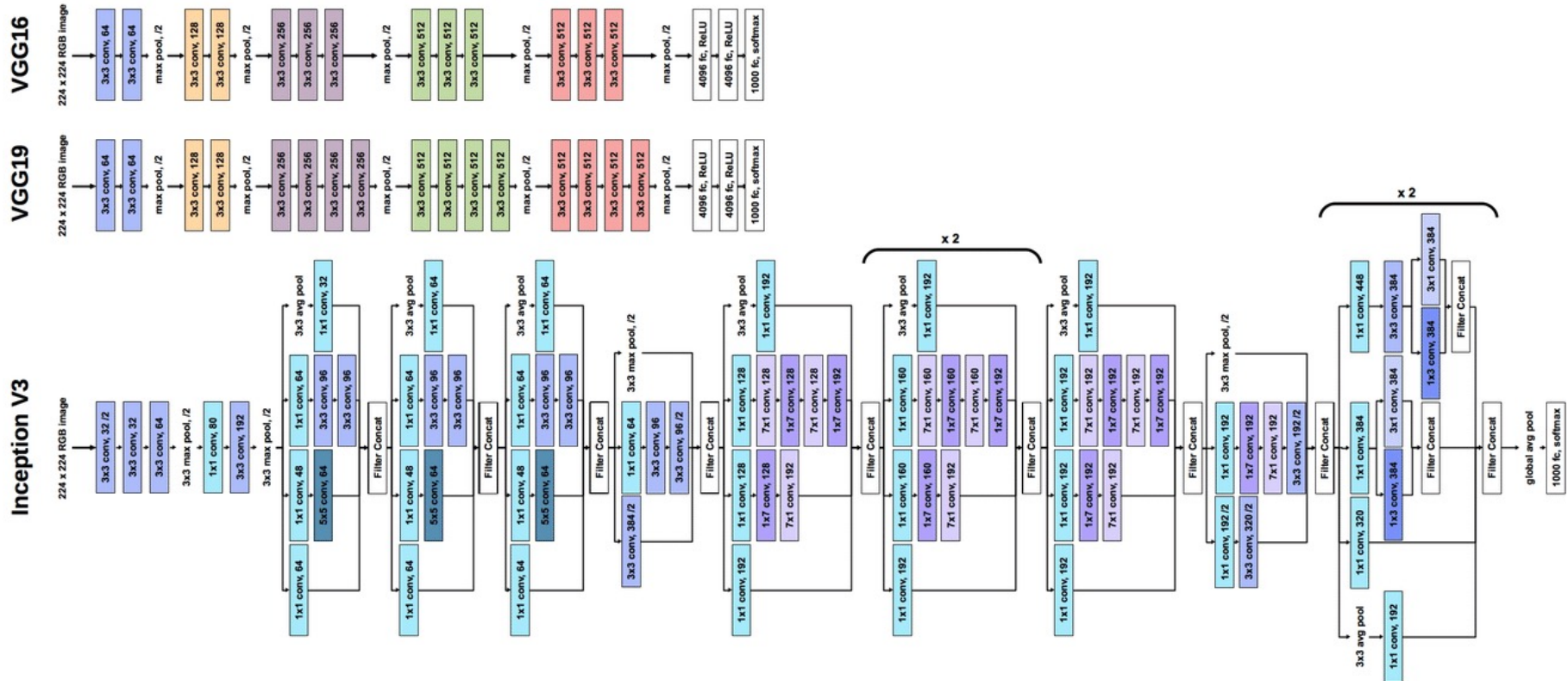
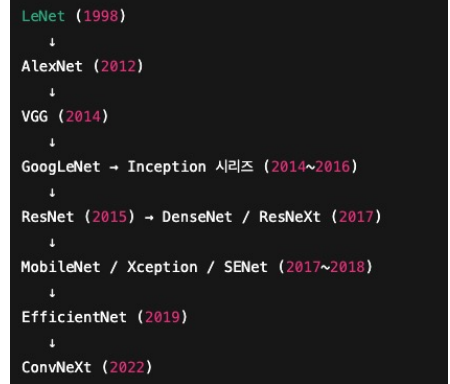
Figure 5. Inception modules where each 5×5 convolution is replaced by two 3×3 convolution, as suggested by principle 3 of Section 2.

```

LeNet (1998)
↓
AlexNet (2012)
↓
VGG (2014)
↓
GoogLeNet → Inception 시리즈 (2014~2016)
↓
ResNet (2015) → DenseNet / ResNeXt (2017)
↓
MobileNet / Xception / SEnet (2017~2018)
↓
EfficientNet (2019)
↓
ConvNeXt (2022)
    
```

Computer Vision

- 시각 인지 기술의 큰 흐름
 - CNN 아키텍처 기반 연구 (2012~)
 - Inception(2014)



V3는 factorized convolution 사용을 확인 가능

[illegible]

- ```
LeNet (1998)
↓
AlexNet (2012)
↓
VGG (2014)
↓
GoogLeNet → Inception 시리즈 (2014~2016)
↓
ResNet (2015) → DenseNet / ResNeXt (2017)
↓
MobileNet / Xception / SEnet (2017~2018)
↓
EfficientNet (2019)
↓
ConvNeXt (2022)
```



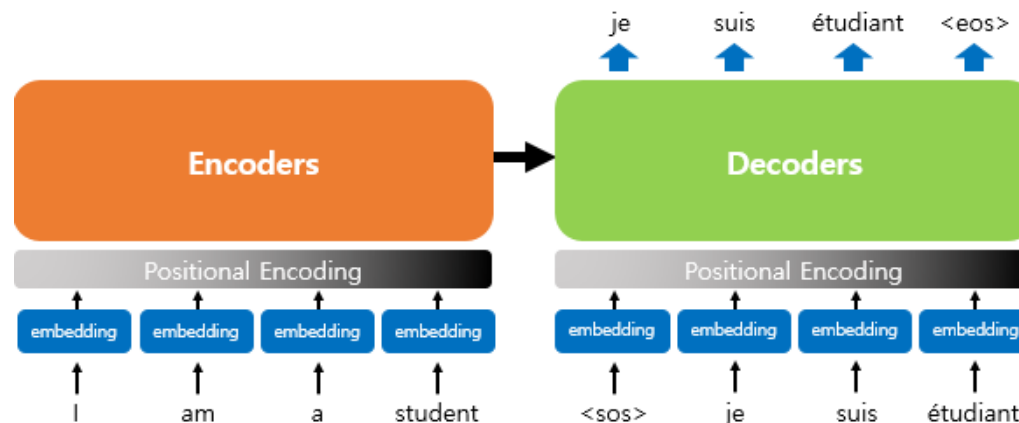
# Computer Vision

- 시각 인지 기술의 큰 흐름

- Vision Transformer 아키텍처 기반 연구 (2021~)
  - 자연어 분야에서 발표된 Transformer (2017) 아키텍처를 비전 분야 아키텍처로 도입한 사례

- Transformer (2017, Attention is all you need)

- RNN을 사용하지 않고, 인코더-디코더 구조를 설계하였음에도 번역 성능이 RNN보다 우수함
  - Seq2Seq 방법론이 인코더-디코더 구조로 입력 시퀀스 정보를 손실
- RNN이 유용했던 이유는 단어의 위치에 따라 단어를 순차적으로 입력받아서 처리하는 특성 때문임
- 트랜스포머는 단어의 위치 정보를 얻기 위해서 각 단어의 임베딩 벡터에 위치 정보들을 더하여 모델의 입력으로 사용하는데 이를 포지셔널 인코딩 이라고 함



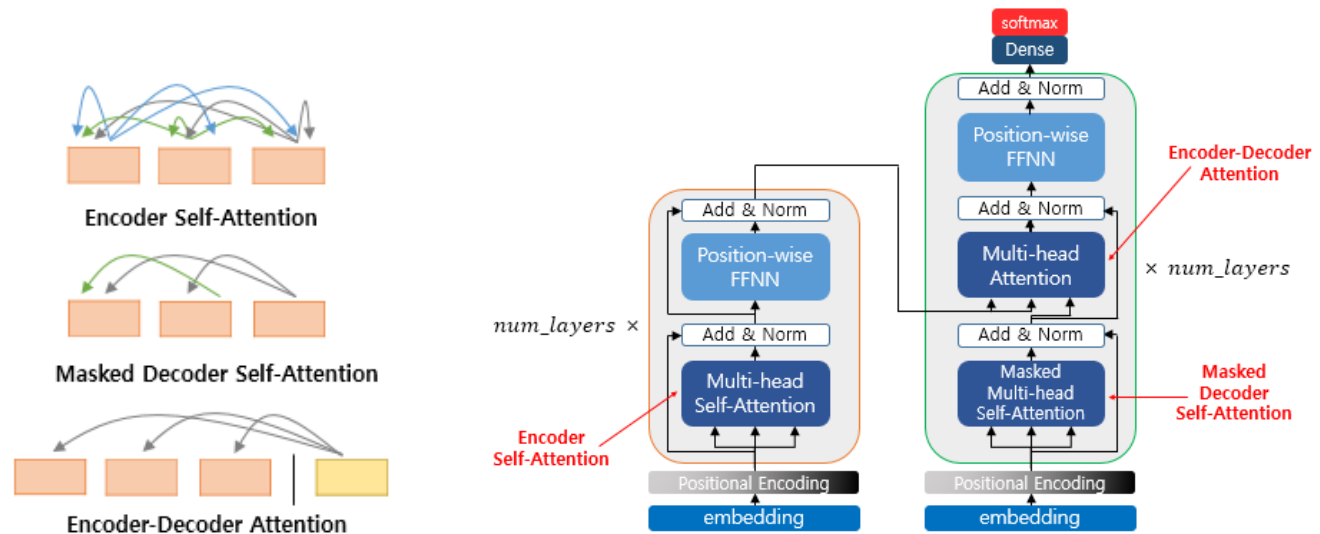


# Computer Vision

- 시각 인지 기술의 큰 흐름

- Transformer (2017, Attention is all you need)

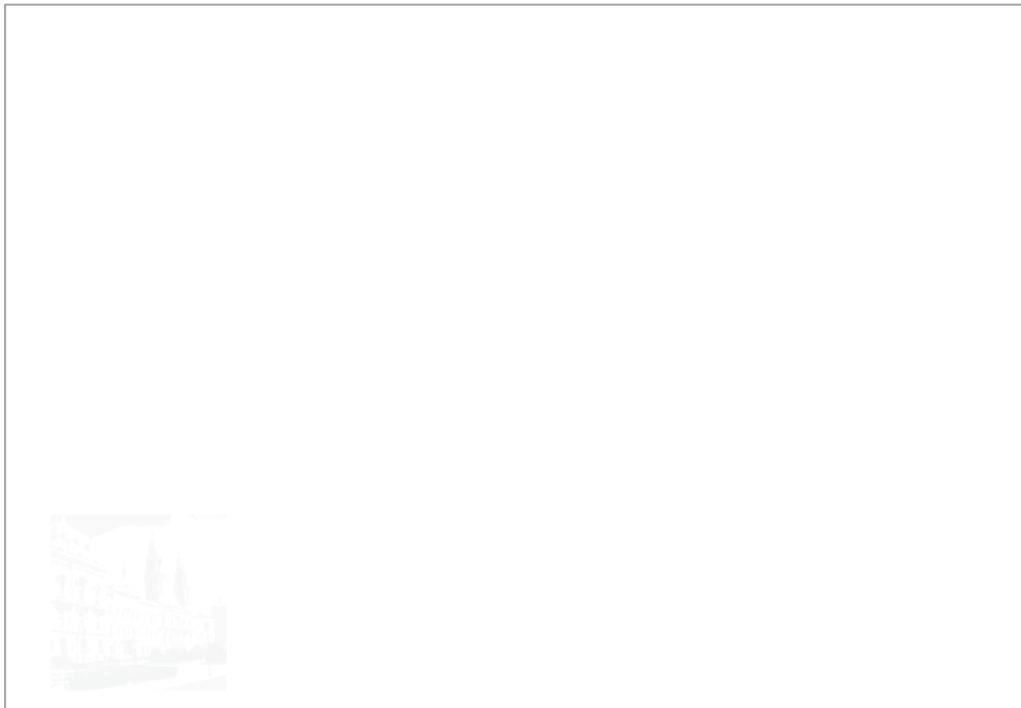
- 첫번째 그림인 셀프 어텐션은 인코더에서 이루어짐
    - 두번째 그림인 셀프 어텐션과 세번째 그림인 인코더-디코더 어텐션은 디코더에서 이루어짐
      - 셀프 어텐션은 본질적으로 Query, Key, Value가 동일한 경우를 말함
      - 주의할 점은 Query, Key 등이 같다는 것은 벡터의 값이 같다는 것이 아니라 벡터의 출처가 같다는 의미
    - 세번째 그림 인코더-디코더 어텐션에서는 Query가 디코더의 벡터인 반면에 Key와 Value가 인코더의 벡터이므로 셀프 어텐션이라고 부르지 않음



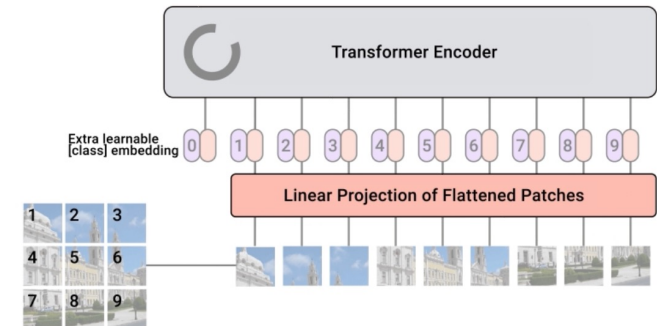
# Computer Vision

- 시각 인지 기술의 큰 흐름

- Vision Transformer 아키텍처 기반 연구 (2021~)
  - 자연어 분야에서 발표된 Transformer (2017) 아키텍처를 비전 분야 아키텍처로 도입한 사례
    - 이미지도 자연어처럼 '토큰'으로 나뉘어서 Transformer로 처리하자는 아이디어



ViT 모델 입력을 이해해보자.



Transformer 처럼 이미지를 패치 단위로 끊어, Sequence Data 형태로 만들어 줌 (Embedded Patches)

- 우선 Input 이미지를 정해진 패치 사이즈로 맞게 나눠줌

- 그 후 CLS Token과 Concat 시키고, 각 패치에 Positioning Embedding을 더해줌

# Computer Vision

- 시각 인지 기술의 큰 흐름

- Vision Transformer 아키텍처 기반 연구 (2021~)
  - 자연어 분야에서 발표된 Transformer (2017) 아키텍처를 비전 분야 아키텍처로 도입한 사례
    - 이미지도 자연어처럼 ‘토큰’으로 나눠서 Transformer로 처리하자는 아이디어
  - 백본의 발전은 순수 Transformer 구조 도입 (ViT), 데이터 효율성 향상 (DeiT), 로컬 구조 보강 (Swin, CvT, T2T), 대규모사전학습 (ViT, MAE, BEiT) 으로 진화함

```
ViT (2020)
↓
DeiT / T2T / CvT / PVT / Swin (2021)
↓
CaiT / CrossViT / ViTDet / BEiT / MAE (2022)
↓
SwinV2 / EVA / InternViT (2023~2024)
```

ViT: 순수 transformer 기반 비전 모델의 시작

DeiT: distillation으로 데이터 효율 개선

Swin: 지역성 확보, CNN 대체 백본\*\*

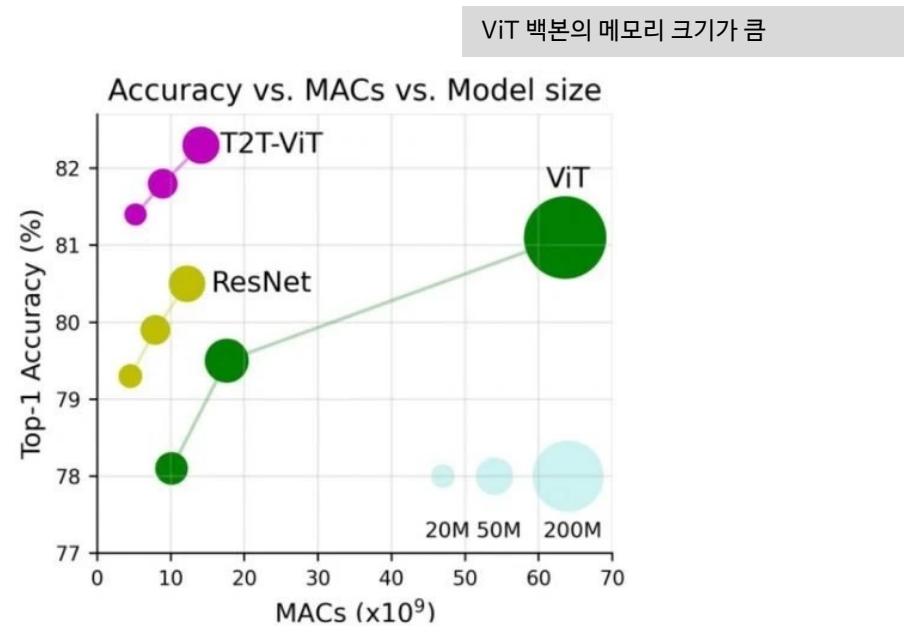
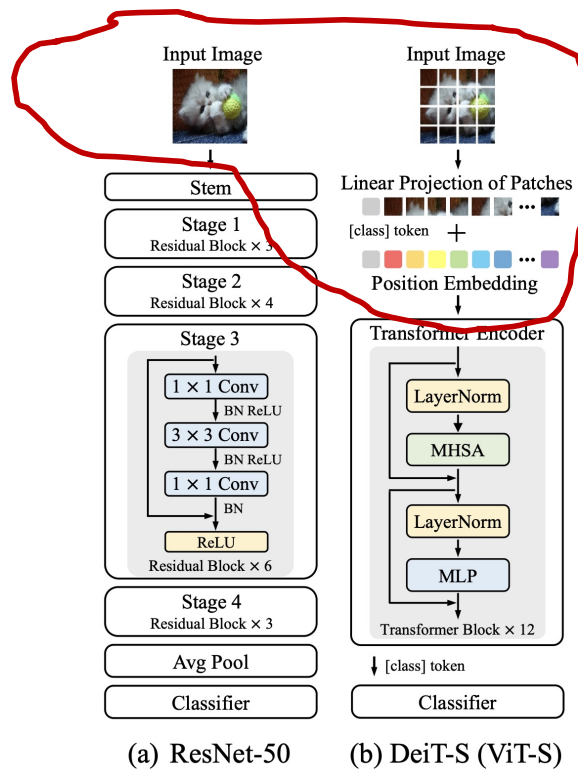
CaiT: attention 설계, 깊은 ViT 학습 가능

ViTDet: ViT를 detection 백본으로 최적화

SwinV2: 초대형 이미지에서도 학습 가능

# Computer Vision

- 시각 인지 기술의 큰 흐름
  - Vision Transformer 아키텍처 기반 연구 (2021~)
    - CNN vs ViT



# Computer Vision

- 시각 인지 기술의 큰 흐름

- VLM (Vision Language Model) 기반 연구 (2021~)

- VLM 모델이란, 이미지와 텍스트를 동시에 이해할 수 있도록 학습된 멀티모달 모델
    - VLM 모델은 대부분 Transformer에 기반하고 있으며, (1) 일반적으로 이미지 모델, (2) 텍스트 모델 그리고 (3) 두 개의 모달리티를 결합하는 모듈 이렇게 세 가지의 요소로 구성되어 있음
    - 대표적인 연구 : CLIP(2021, openAI), BLIP(2022, salesforce)
      - CLIP(Contrastive Language-Image Pre-training), 빠르고 효율적인 매칭 및 검색 작업에 적합
      - BLIP(Bootstrapping Language-Image Pre-training), 생성 작업이 필요한 복합적인 비전-언어에 적합

|       | CLIP                                                   | BLIP                                                               |
|-------|--------------------------------------------------------|--------------------------------------------------------------------|
| 개발    | OpenAI(2021)                                           | Salesforce(2022)                                                   |
| 목적    | 텍스트와 이미지를 같은 벡터 공간에서 매핑하여 둘 사이의 이미적 유사도를 계산            | 이미지-텍스트 이해뿐만 아니라, 텍스트 생성(설명, 질문 응답 등)까지 가능                         |
| 훈련방식  | 대규모 이미지-텍스트 쌍을 사용하여 대조 학습(contrastive learning)        | 이미지-텍스트 매칭(ITM) → CLIP 과 유사한 정렬 학습, 이미지 조건 언어 모델링 → 주어진 이미지로 캡션 생성 |
| 출력    | 이미지 임베딩, 텍스트 임베딩<br>- 두 벡터의 코사인 유사도 계산                 | 자연어 문장(캡션, 대화, 설명 등)                                               |
| 주요 활용 | 이미지 검색(텍스트->이미지 매칭), 제로샷 분류(텍스트 프롬프트 기반), 멀티모달 검색 및 정렬 | 이미지 캡셔닝(자동 설명 생성), 비주얼 질문 응답(VQA)                                  |

# Computer Vision

- 시각 인지 기술의 큰 흐름
  - VLM (Vision Language Model) 기반 연구 (2021~)
    - 대표적인 연구 : CLIP(2021, openAI), BLIP(2022, salesforce)



CLIP 을 통해,  
image feature와 text feature와  
가장 유사한 녀석을 정답으로 결정

주어진 텍스트 리스트는  
(고양이, 강아지, 자전거 타는 사람)



BLIP을 통해

추출한 캡션 embedding feature와  
주어진 텍스트 리스트의 text feature 중  
가장 유사한 녀석을 정답으로 결정



# Computer Vision

## ■ 시각 인지 기술의 큰 흐름

### ■ VLM (Vision Language Model) 기반 연구 (2021 ~)

#### ■ CLIP(2021, openAI)

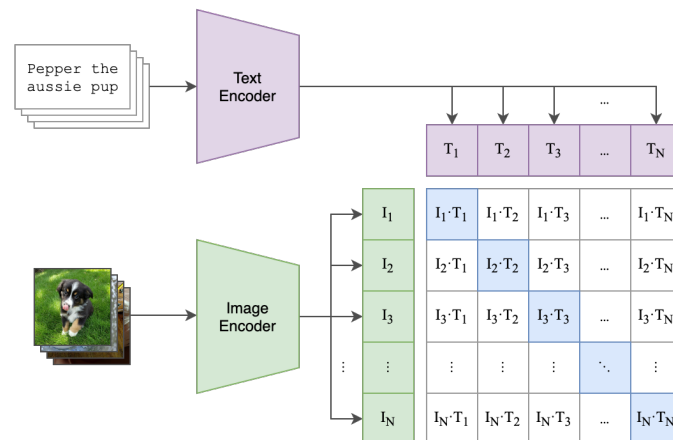
#### ■ Contrastive Learning(대조 학습) 방식 사용:

- 같은 의미의 텍스트-이미지 쌍을 가깝게(Low distance),
- 다른 의미의 텍스트-이미지 쌍을 멀게(High distance) 학습하는 방식

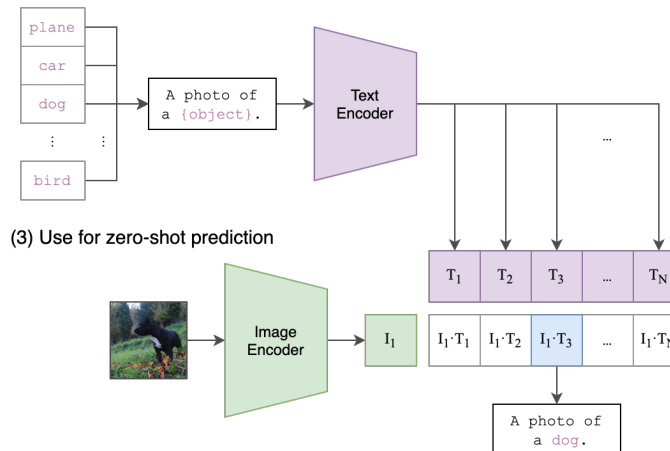
#### ■ Zero-shot 학습 가능:

- CLIP은 사전에 학습된 데이터만으로도 강력한 성능을 발휘
- 예를 들어, 새로운 이미지 카테고리 위한 분류 모델을 학습하지 않아도, "고양이", "강아지" 등의 텍스트와 이미지를 비교하여 분류 가능

(1) Contrastive pre-training



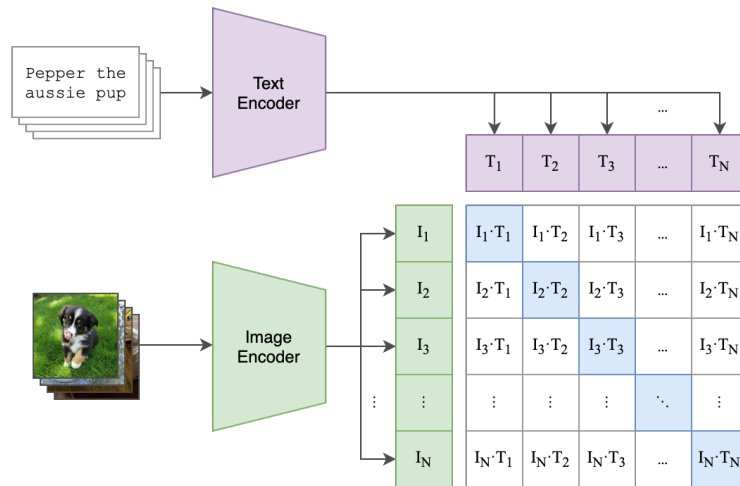
(2) Create dataset classifier from label text



# Computer Vision

- 시각 인지 기술의 큰 흐름
  - VLM (Vision Language Model) 기반 연구 (2021 ~)
    - CLIP(2021, openAI)

(1) Contrastive pre-training



- Vision Encoder (이미지 인코더)
  - CNN(ResNet-50/101) 또는 Vision Transformer (ViT) 기반
  - 입력된 이미지를 벡터 형태로 변환
- Text Encoder (텍스트 인코더)
  - Transformer 기반 모델 사용 (GPT 아키텍처와 유사)
  - 입력된 텍스트를 벡터 형태로 변환
- 핵심 아이디어:
  - 이미지와 텍스트를 같은 차원의 벡터 공간에 매핑한 후, 의미가 비슷한 경우 벡터 거리가 가까워지도록 학습하는 방식 (contrastive learning, 대조학습)

## 예제

이미지 1: 🐶 (강아지 사진)

이미지 2: 🚗 (자동차 사진)

텍스트 1: "A cute dog"

텍스트 2: "A fast car"

👉 CLIP은 🐶 ↔ "A cute dog"의 유사도를 높이고, 🚗 ↔ "A fast car"의 유사도를 낮추는 방향으로 학습됨

# Computer Vision

## ■ 시각 인지 기술의 큰 흐름

- VLM (Vision Language Model) 기반 연구 (2021 ~)
  - CLIP(2021, openAI)
    - CLIP 모델 아키텍처 개요
      - Vision Encoder (ResNet or Vision Transformer 기반)
      - Text Encoder (Transformer 기반, GPT-like 구조)
      - Projection Head (이미지와 텍스트 임베딩을 동일한 차원으로 변환)
      - Contrastive Loss (이미지-텍스트 매칭 학습)

BERT는 각 입력 토큰마다 벡터(=hidden state)를 출력합니다.

예를 들어 입력이 [CLS] 나는 고양이를 좋아해 [SEP] 라면:

| 토큰    | 출력 벡터 (예시)                |
|-------|---------------------------|
| [CLS] | 문장 전체를 표현하는 벡터 (768차원 벡터) |
| 나는    | (768차원 벡터)                |
| 고양이를  | (768차원 벡터)                |
| 좋아해   | (768차원 벡터)                |
| [SEP] | 문장 구분자 (768차원 벡터)         |

## CLIP의 이미지 인코더 : ViT 기반

```
import torch
import torch.nn as nn
from torchvision.models import vit_b_32

class VisionTransformer(nn.Module):
 def __init__(self, embed_dim=512):
 super().__init__()
 self.model = vit_b_32(pretrained=True) # 미리 학습된 ViT 사용
 self.fc = nn.Linear(768, embed_dim) # 최종 출력 차원을 512로 변환

 def forward(self, images):
 x = self.model(images) # 이미지 인코딩
 x = self.fc(x) # FC 레이어를 통해 차원 변환
 return x
```

## CLIP의 텍스트 인코더 : Transformer 기반 (Bert 계열)

```
from transformers import RobertaModel

class TextEncoder(nn.Module):
 def __init__(self, embed_dim=512):
 super().__init__()
 self.model = RobertaModel.from_pretrained("roberta-base") # Pretrained Transformer
 self.fc = nn.Linear(768, embed_dim) # 차원 축소

 def forward(self, input_ids, attention_mask):
 x = self.model(input_ids=input_ids, attention_mask=attention_mask)
 x = x.last_hidden_state[:, 0, :] # CLS 토큰 사용
 x = self.fc(x) # FC 레이어
 return x
```

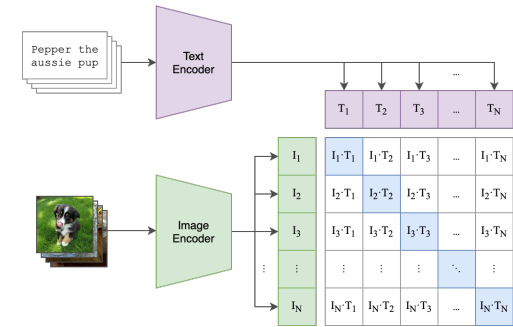
Bert의 출력 중 [CLS 토큰]만 사용

# Computer Vision

## ■ 시각 인지 기술의 큰 흐름

- VLM (Vision Language Model) 기반 연구 (2021 ~)
  - CLIP(2021, openAI)
    - CLIP 모델 아키텍처 개요
      - Vision Encoder (ResNet or Vision Transformer 기반)
      - Text Encoder (Transformer 기반, GPT-like 구조)
      - Projection Head (이미지와 텍스트 임베딩을 동일한 차원으로 변환)
      - Contrastive Loss (이미지-텍스트 매칭 학습)

(1) Contrastive pre-training



## Projection Head (공통 임베딩 공간 매핑)

```
class ProjectionHead(nn.Module):
 def __init__(self, embed_dim=512, proj_dim=512):
 super().__init__()
 self.projection = nn.Linear(embed_dim, proj_dim)
 self.scale = nn.Parameter(torch.ones(1) * 0.07)

 def forward(self, x):
 x = self.projection(x)
 x = x / x.norm(dim=-1, keepdim=True) # L2 정규화
 return x * self.scale
```

# 512 -> 512 (동일 차원 변환)

# scaling (Temperature)

### 3. 온도( $\tau$ )의 의미

| $\tau$ 값        | Softmax 분포             | 효과                     |
|-----------------|------------------------|------------------------|
| 작을수록 (예: 0.01)  | 더 날카로운 (한 쌍이 압도적으로 높음) | 모델이 "자신감 있게" 특정 짝을 선택  |
| 클수록 (예: 1.0 이상) | 원만한 분포 (모든 항목이 비슷)     | 모델이 "예매하게" 여러 쌍에 확률 분산 |

즉,  $\tau$ 는 확률 분포의 sharpness (날카로움)를 조절하는 역할을 합니다.

## Contrastive Learning (대조학습)

```
import torch.nn.functional as F

class CLIPLoss(nn.Module):
 def forward(self, image_features, text_features):
 # 이미지와 텍스트 간 내적 계산
 logits = image_features @ text_features.T # Cosine Similarity
 labels = torch.arange(len(logits)).to(logits.device) # 정답 레이블 생성

 # CrossEntropyLoss 적용 (이미지-텍스트 정답 매칭) # 정답 텍스트와 가장 유사한 이미지 찾도록 학습
 loss = (F.cross_entropy(logits, labels) + F.cross_entropy(logits.T, labels)) / 2
 return loss
```

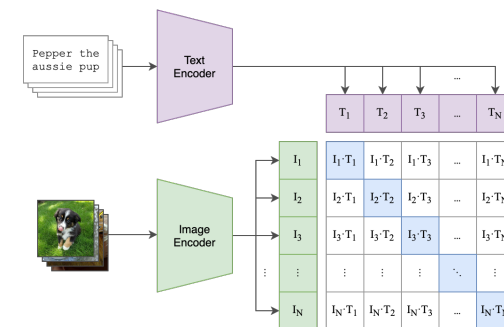
# Computer Vision

- 시각 인지 기술의 큰 흐름
  - VLM (Vision Language Model) 기반 연구 (2021~)
    - CLIP(2021, openAI)
      - CLIP 모델 아키텍처 개요
        - Vision Encoder (ResNet or Vision Transformer 기반)
        - Text Encoder (Transformer 기반, GPT-like 구조)
        - Projection Head (이미지와 텍스트 공통 임베딩 공간으로 변환)
        - Contrastive Loss (이미지-텍스트 매칭 학습)

```
class CLIP(nn.Module):
 def __init__(self, embed_dim=512):
 super().__init__()
 self.vision_encoder = VisionTransformer(embed_dim)
 self.text_encoder = TextEncoder(embed_dim)
 self.vision_proj = ProjectionHead(embed_dim)
 self.text_proj = ProjectionHead(embed_dim)
 self.loss_fn = CLIPLoss()

 def forward(self, images, input_ids, attention_mask):
 image_features = self.vision_proj(self.vision_encoder(images))
 text_features = self.text_proj(self.text_encoder(input_ids, attention_mask))
 loss = self.loss_fn(image_features, text_features)
 return loss, image_features, text_features
```

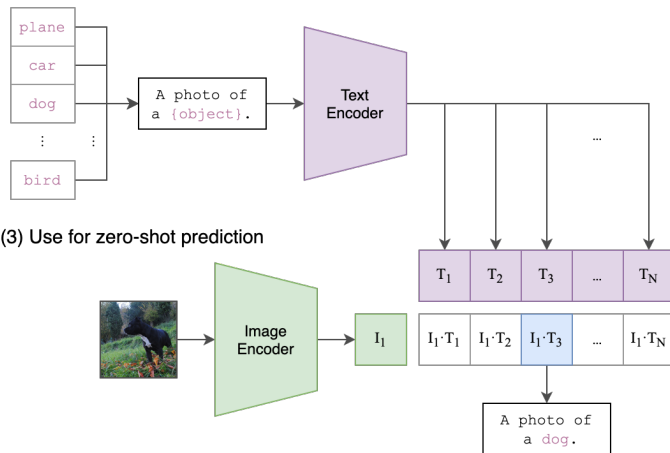
(1) Contrastive pre-training



# Computer Vision

- 시각 인지 기술의 큰 흐름
  - VLM (Vision Language Model) 기반 연구 (2021 ~)
    - CLIP(2021, openAI)
      - CLIP 코드 실행

(2) Create dataset classifier from label text



```
model, preprocess = clip.load("ViT-B/32", device=device)
```

```
이미지 로드 및 변환
image = preprocess(Image.open("dog.jpg")).unsqueeze(0).to(device)
```

```
텍스트 입력
texts = ["a photo of a dog", "a photo of a cat"]
text_tokens = clip.tokenize(texts).to(device)
```

```
이미지 및 텍스트 특징 추출
with torch.no_grad():
 image_features = model.encode_image(image)
 text_features = model.encode_text(text_tokens)
```

Encoder를 projection head 통과한 vector

```
유사도 계산
similarity = (image_features @ text_features.T).softmax(dim=-1)
print(similarity)
```



# Computer Vision

- 시각 인지 기술의 큰 흐름

| Task                 | CNN 기반 연구                                     | ViT기반 연구                               | VLM 기반 연구                                                                           |
|----------------------|-----------------------------------------------|----------------------------------------|-------------------------------------------------------------------------------------|
| Image Classification | ResNet (2015)<br>EfficientNet (2019)          | ViT (2020)<br>DeiT (2021)              | CLIP (OpenAI, 2021)                                                                 |
| Object Detection     | Faster R-CNN (2015)<br>YOLOv3/v4 (2018, 2020) | DETR (2020)<br>DINO (2023)             | OWL-ViT (Google, 2022)<br>Grounding DINO (IDEA, 2024)<br>YOLO-World (Tencent, 2024) |
| Object Segmentation  | Mask R-CNN (2017)<br>DeepLabv3+ (2018)        | Mask2Former (2022)<br>Segmenter (2021) | Grounded-SAM (Meta, 2023)<br>SAM (Meta, 2023)<br>OWL-Seg (Google2024)               |

# Computer Vision

- “시각 인지 기술”별 최신 트렌드
  - Image Classification 최신 트렌드
  - Object Detection 최신 트렌드
  - Object Segmentation 최신 트렌드

**Classification**



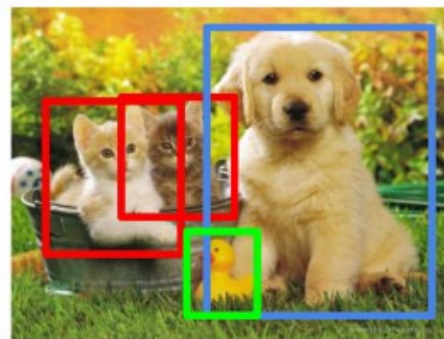
CAT

**Classification  
+ Localization**



CAT

**Object Detection**



CAT, DOG, DUCK

**Instance  
Segmentation**



CAT, DOG, DUCK

Single object

Multiple objects

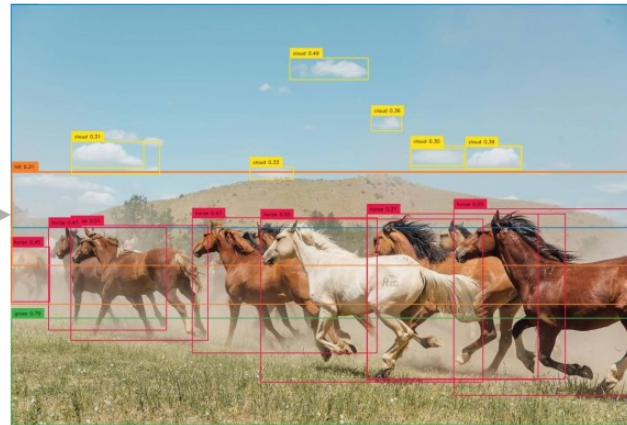
# Computer Vision

- 시각 인지 기술의 큰 흐름

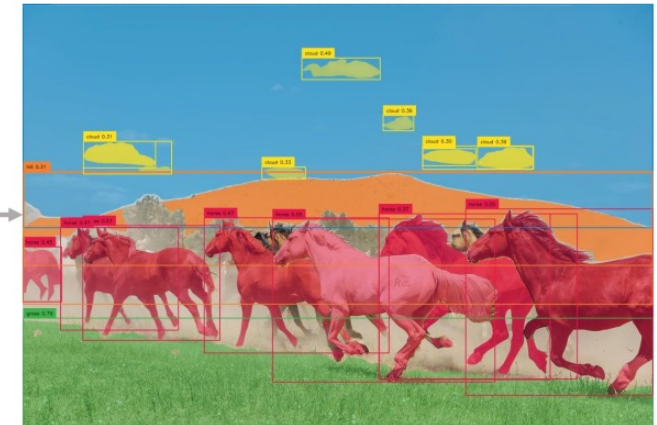
| Task                 | CNN 기반 연구                                     | ViT기반 연구                               | VLM 기반 연구                                                                           |
|----------------------|-----------------------------------------------|----------------------------------------|-------------------------------------------------------------------------------------|
| Image Classification | ResNet (2015)<br>EfficientNet (2019)          | ViT (2020)<br>DeiT (2021)              | CLIP (OpenAI, 2021)                                                                 |
| Object Detection     | Faster R-CNN (2015)<br>YOLOv3/v4 (2018, 2020) | DETR (2020)<br>DINO (2023)             | OWL-ViT (Google, 2022)<br>Grounding DINO (IDEA, 2024)<br>YOLO-World (Tencent, 2024) |
| Object Segmentation  | Mask R-CNN (2017)<br>DeepLabv3+ (2018)        | Mask2Former (2022)<br>Segmenter (2021) | Grounded-SAM (Meta, 2023)<br>SAM (Meta, 2023)<br>OWL-Seg (Google2024)               |



Text Prompt:  
“Horse. Clouds. Grasses. Sky. Hill.”



Grounding DINO:  
Detect Everything



Grounded-SAM:  
Detect and Segment Everything

# Computer Vision

- Image Classification 최신 트렌드

| Task                 | CNN 기반 연구                            | ViT기반 연구                  | VLM 기반 연구           |
|----------------------|--------------------------------------|---------------------------|---------------------|
| Image Classification | ResNet (2015)<br>EfficientNet (2019) | ViT (2020)<br>DeiT (2021) | CLIP (OpenAI, 2021) |

- 모델 경량화

- 모바일 및 엣지 디바이스에서 효율적으로 동작하는 MobileNet, EfficientNet등이 지속적으로 개선

- Vision Transformer(ViT) 아키텍처의 확산

- 이미지를 패치로 분할하여 Transformer에 입력, 대규모 데이터셋에서 뛰어난 성능을 보임
    - Swin Transformer와 같이 연산 효율성과 정확도를 동시에 향상시키는 연구들도 연구됨

- 제로샷 및 오픈셋 분류

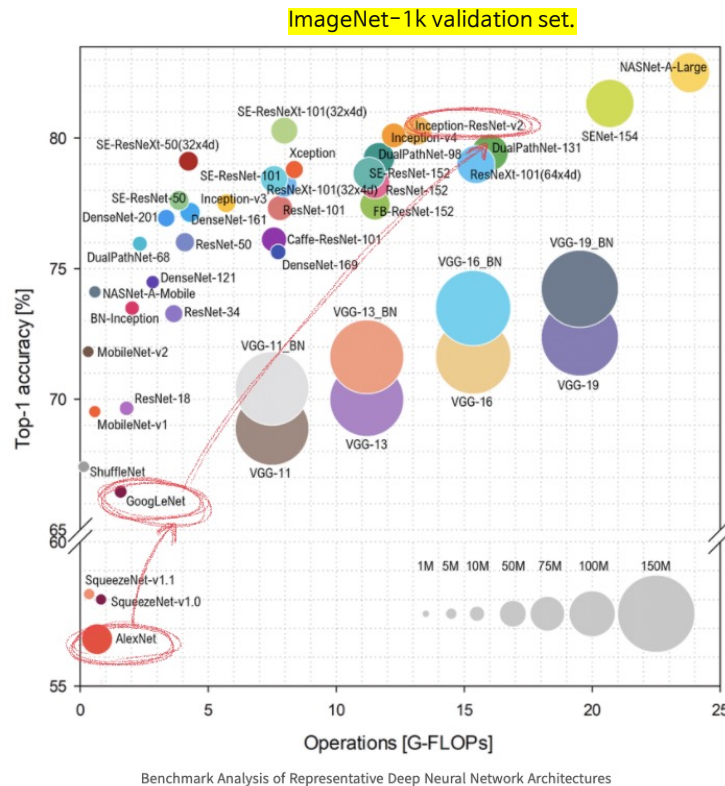
- CLIP 기반 모델 : 텍스트와 이미지를 공동 임베딩하여 새로운 클래스를 텍스트 설명만으로 인식 가능
    - 오픈셋 인식: 알려지지 않은 클래스를 감지하고 새롭게 분류하는 연구가 활발히 진행

# Computer Vision

- Image Classification 최신 트렌드

- 모델 경량화

- 모바일 및 엣지 디바이스에서 효율적으로 동작하는 MobileNet, EfficientNet등이 지속적으로 개선



- ✓ 연산 효율성
  - ✓ MobileNet, ShuffleNet
- ✓ 정확도 유지하며, 모델 파라미터 줄이기
  - ✓ SqueezeNet

G-FLOPS : GPU Floating point Operations Per Second. 초당 부동소숫점 연산량



# Computer Vision

- Image Classification 최신 트렌드

- Vision Transformer(ViT) 아키텍처의 확산

- 이미지를 패치로 분할하여 Transformer에 입력, 대규모 데이터셋에서 뛰어난 성능을 보임
    - Swin Transformer와 같이 연산 효율성과 정확도를 동시에 향상시키는 연구들도 연구됨



ConvNeXt는 전통적인 CNN을 Transformer 수준의 성능과 구조적 단순함으로 재설계한 모델임



# Computer Vision

- Object Detection 최신 트렌드
  - YOLO 시리즈의 지속적 발전을 통한 실시간 객체 검출
    - YOLOv10: 효율적인 백본 네트워크와 새로운 검출 헤드 도입
    - YOLOv11: C3K2블록, SPPF(Spatial Pyramid Pooling-FAST), C2PSA(Convolutional Block with Parallel Spatial Attention)등의 아키텍처 개선
  - Transformer 기반 검출기 확산
    - DETR(Detection Transformer)
      - CNN없이 Transformer만으로 객체 검출을 수행하며, 복잡한 후처리 없이 높은 정확도 보임
  - 소량 학습 (Few-Shot Learning) 기반 객체 검출
    - 라벨링된 데이터가 부족한 상황에서 새로운 객체를 빠르게 학습하는 Few-Shot Object Detection(FSOD)이 주목 받는 중
      - 메타러닝(Meta Learning), 프로토타입 기반 학습 등
  - 제로샷(Zero-shot) 기반 객체 검출 → Open-Vocabulary Object Detection
    - 사전 정의된 클래스가 아니라, 텍스트 프롬프트로 객체를 탐지하는 방식
    - 학습 클래스가 아니더라도, 단어만 주어진다면 탐지 가능 (예. drone이라는 객체를 탐지해줘)
    - 텍스트 임베딩을 사용해 이미지에서 해당 객체 탐지

# Computer Vision

## Object Detection 최신 트렌드

### Object Detection Milestones

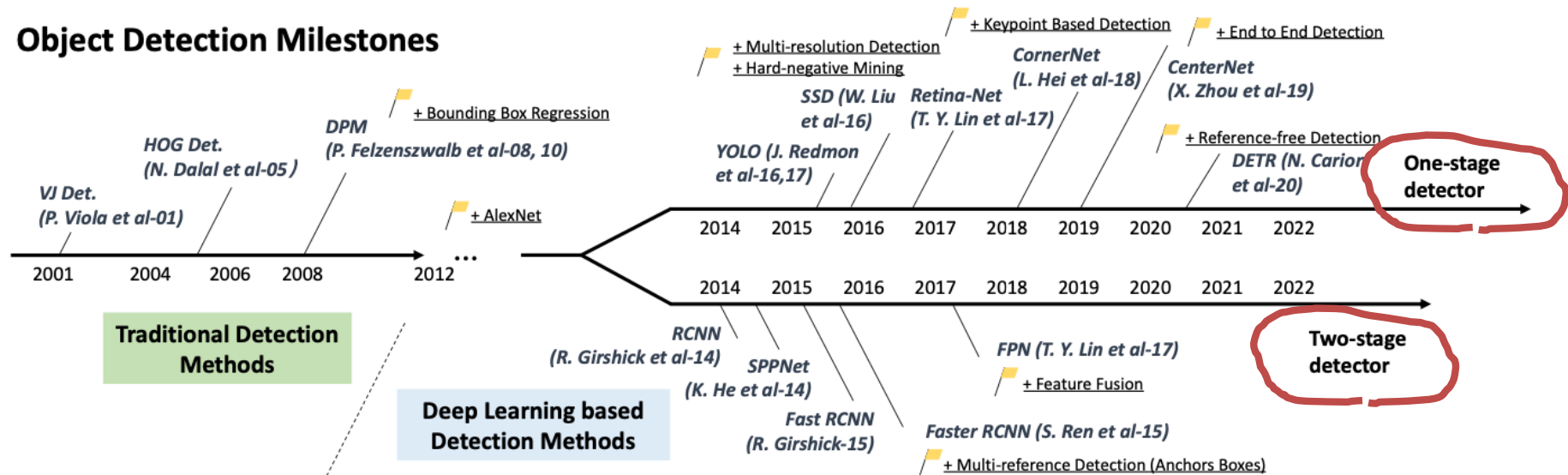


Fig. 2: A road map of object detection. Milestone detectors in this figure: VJ Det. [10, 11], HOG Det. [12], DPM [13-15], RCNN [16], SPPNet [17], Fast RCNN [18], Faster RCNN [19], YOLO [20-22], SSD [23], FPN [24], Retina-Net [25], CornerNet [26], CenterNet [27], DETR [28].

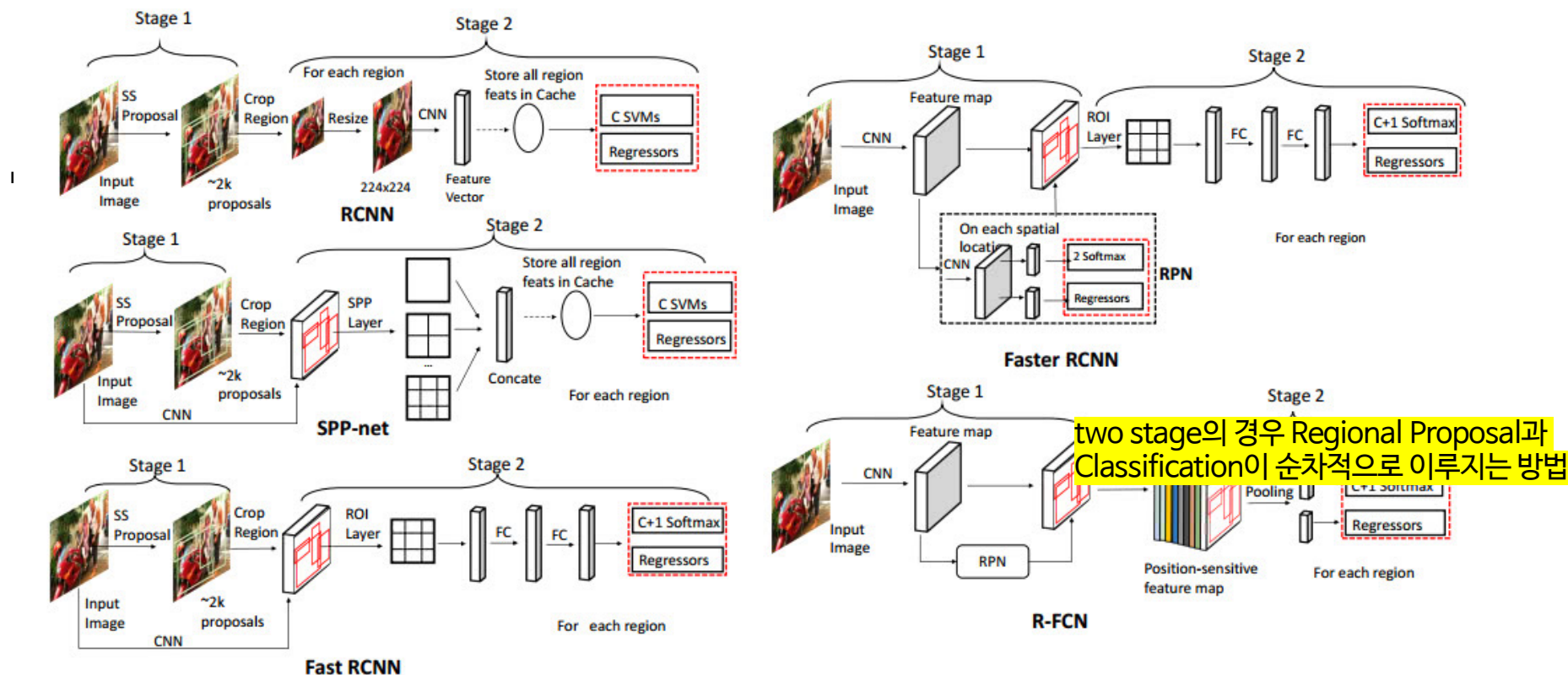


Figure 4: Overview of different two-stage detection frameworks for generic object detection. Red dotted rectangles denote the outputs that define the loss functions.

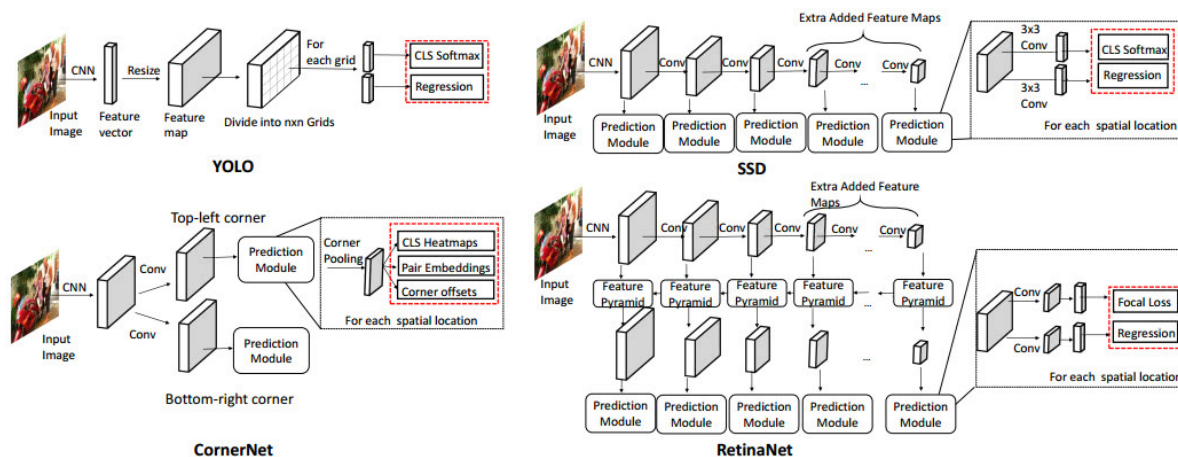


Figure 5: Overview of different one-stage detection frameworks for generic object detection. Red rectangles denotes the outputs that define the objective functions.

# Computer Vision

## ▪ Object Detection 최신 트렌드

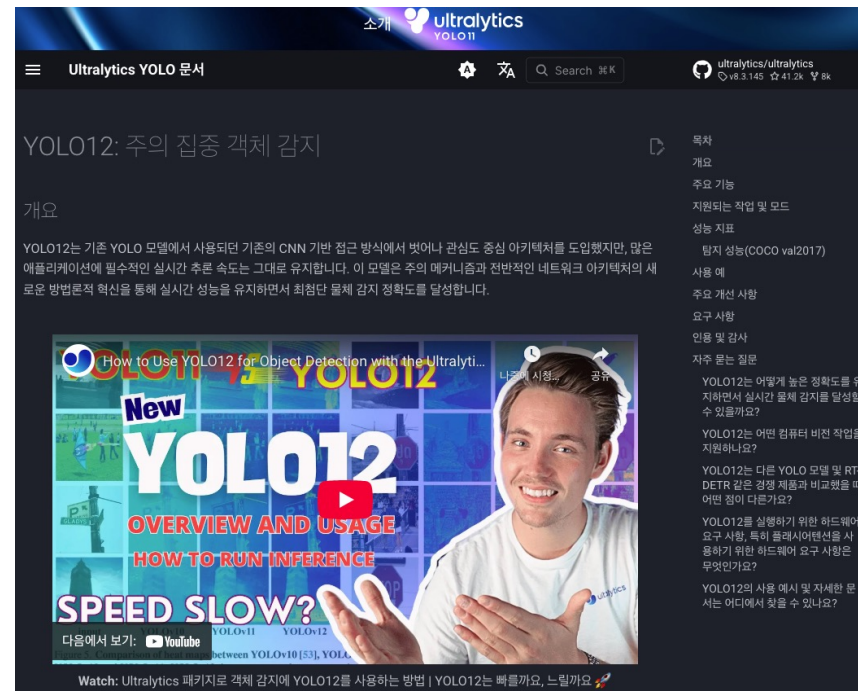
### ▪ YOLO 시리즈의 지속적 발전을 통한 실시간 객체 검출

- YOLOv10: 효율적인 백본 네트워크와 새로운 검출 헤드 도입
- YOLOv11: C3K2블록, SPPF(Spatial Pyramid Pooling-FAST), C2PSA(Convolutional Block with Parallel Spatial Attention)등의 아키텍처 개선

| 항목     | 내용                                                                  |
|--------|---------------------------------------------------------------------|
| 회사명    | Ultralytics                                                         |
| 설립자    | Glenn Jocher<br>(전 Tesla AI 엔지니어)                                   |
| 주요 제품  | YOLOv5, YOLOv8,<br>Ultralytics HUB, YOLO-World                      |
| 본사 위치  | 미국 텍사스 주 오스틴                                                        |
| 웹사이트   | <a href="https://ultralytics.com">https://ultralytics.com</a>       |
| GitHub | <a href="https://github.com/ultralytics">github.com/ultralytics</a> |

Ultralytics는 YOLO(You Only Look Once) 시리즈의 최신 버전 개발과 보급을 주도하고 있는 비전 AI 전문 기업이며, YOLO 기술을 기반으로 한 오픈소스 프레임워크 및 상업용 제품군을 제공합니다.

특히 YOLOv5~v8, 그리고 최근에는 YOLO-World까지 주도적으로 개발/유지하고 있음





# Computer Vision

## ▪ Object Detection 최신 트렌드

### ▪ Transformer 기반 검출기 확산

- DETR(Detection Transformer): CNN없이 Transformer만으로 객체 검출을 수행하며, 복잡한 후처리 없이 높은 정확도
- Swin Transformer: Shifted Window 방식을 통해 연산 효율성과 정확도를 동시에 향상시킨 모델

Transformer 기반 Object Detection 대표 연구 흐름

| 모델                           | 발표 연도 | 주요 특징                                                                                           |
|------------------------------|-------|-------------------------------------------------------------------------------------------------|
| <b>**DETR (Facebook AI)</b>  | 2020  | 최초 Transformer 기반 검출기, Anchor-free, NMS-free 기법 사용, 학습 속도 느림                                    |
| Deformable DETR (MSRA)       | 2020  | Sparse attention → 학습 속도 개선멀티스케일 대응 우수<br>→ 후속연구들이 Swin-Transformer를 백본으로 많이들 사용함               |
| Sparse DETR (KaKao Brain)    | 2021  | Cross-attention 영역을 제한하여 효율성 확보                                                                 |
| Anchor DETR (megvii)         | 2021  | Anchor-free 한계를 보완하기 위해 query 초기화에 anchor 개념 도입                                                 |
| UP-DETR (Tencent)            | 2020  | self-supervised pretraining을 DETR에 결합추가 label 없이 학습                                             |
| Efficient DETR (megvii)      | 2021  | 디코더 구조 최적화로 연산량 감소                                                                              |
| <b>DINO (IDEA)</b>           | 2022  | DINO: Self-Distillation with No Labels 의 약자<br>자기지도 학습 방식으로 교사-학생 네트워크 구조를 이용해 라벨 없이 시각 표현을 학습함 |
| <b>**OWL-ViT (Google)</b>    | 2022  | CLIP 기반 zero-shot detection 가능Open-Vocabulary 탐지                                                |
| <b>Grounding DINO (IDEA)</b> | 2023  | 언어 기반 open-world 탐지텍스트 쿼리로 객체 탐지                                                                |
| <b>YOLO-World (Tencent)</b>  | 2024  | YOLOv8 + Transformer 기반 프롬프트 탐지CLIP 활용                                                          |

# Computer Vision

- Object Detection 최신 트렌드

- 소량 학습 (Few-Shot Learning) 기반 객체 검출

- 라벨링된 데이터가 부족한 상황에서 새로운 객체를 빠르게 학습하는 Few-Shot Object Detection (FSOD)이 주목 받는 중
    - 메타러닝 (Meta Learning), 프로토타입 기반 학습 등
      - 메타러닝 : 소수 샷 환경에 빠르게 적응하도록 학습
      - 프로토타입 기반 학습 : 클래스마다 대표 임베딩(프로토타입)을 만들고, RoI 임베딩과의 거리로 분류

| 항목        | Few-Shot Object Detection (FSOD)                            | Zero-Shot Object Detection (ZSOD)         |
|-----------|-------------------------------------------------------------|-------------------------------------------|
| 정의        | 소수의 라벨된 이미지 (예: 1~10장)만 보고 새로운 클래스 탐지                       | 라벨된 이미지 전혀 없이, 언어 정보 또는 외부 지식 기반으로 탐지     |
| 학습 대상 클래스 | 지원 (base) + 소량의 지원 샘플 (novel)                               | 학습 시 전혀 등장하지 않은 클래스 (unseen)              |
| 전제 조건     | novel class에 대해 몇 장의 GT 라벨된 예제 존재<br>예) K-shot : K=10,20,30 | 텍스트 설명, 속성, 임베딩 등 외부 지식 존재                |
| 데이터 요구    | 지원 클래스 수십 장의 이미지 필요                                         | 이미지-텍스트 쌍 혹은 대규모 언어 임베딩                   |
| 주요 기술     | meta-learning, metric learning, prototype matching          | VLM, embedding alignment, prompt learning |
| 대표 모델     | Meta R-CNN, Prototypical R-CNN                              | OWL-ViT, Grounding DINO                   |



# Computer Vision

## ▪ Object Detection 최신 트렌드

### ▪ 제로샷(Zero-shot) 기반 객체 검출

- YOLO-World (2024년 1월), [논문]
- Grounding DINO (2023년 4월), [논문]
- OWL-ViT (2022년 3월), [논문]

*Vision Transformer for Open-World Localization, ECCV 2022*

| 항목       | YOLO-World              | Grounding DINO         | OWL-ViT                          |
|----------|-------------------------|------------------------|----------------------------------|
| 기반 구조    | YOLOv8 + CLIP           | DINO + CLIP            | ViT + CLIP-style joint embedding |
| 입력 방식    | 텍스트 프롬프트                | 텍스트 프롬프트               | 텍스트 프롬프트                         |
| 주요 특징    | 실시간 처리, 경량              | 고정밀 탐지                 | 텍스트-비전 정렬 기반 고정밀 탐지              |
| 실시간성     | 빠름 (~50 FPS)            | 느림 (~15 FPS)           | 중간 (~20 FPS)                     |
| 정확도      | 중간 (실시간 대비 우수)          | 매우 높음                  | 높음                               |
| 프롬프트 정교함 | 긴 텍스트 표현이나 복합 표현 탐지 어려움 | 문장 수준 이해 가능            | 단어~문장 모두 대응                      |
| 세분화 연동   | YOLO-World + Seg        | Grounded-SAM           | 분류/탐지 위주                         |
| 제로샷 학습   | CLIP 기반                 | CLIP 기반 + 언어 쿼리 구성 강화  | *CLIP-style pretrained           |
| 실용성      | 모바일/엣지 적합               | 연구/서버용                 | 연구/클라우드용                         |
| 커스터마이징   | 연구자가 커스터마이징 하기 용이       | fine-tuning 커스터마이징 어려움 | 연구자가 커스터마이징 하기 용이                |

\*Clip-Style 의 의미 : CLIP을 그대로 쓴다는 뜻이 아니라, 이미지-텍스트 대조 학습 방식을 그대로 계승한다는 의미

# Computer Vision

- Object Segmentation 최신 트렌드 (요약)

- 트랜스포머 기반 세분화 모델의 부상

- 트랜스포머 아키텍처는 객체 세분화(segmentation) 분야에서도 주목받고 있음
    - Mask2Former(CVPR2022): 다양한 세분화 작업(semantic, instance, panoptic)을 통합하여 처리할 수 있는 모델로 높은 정확도를 보임

- 멀티모달 및 프롬프트 기반 세분화

- [Segment Anything Model \(SAM\)](#) (2023): Meta에서 개발한 모델, 홈페이지에서 데모 제공, 다양한 프롬프트(텍스트 혹은 클릭한 지점, 박스)를 통해 세분화 수행
    - Grounded-SAM (2024): (1)Grounding DINO과 (2)SAM을 결합한 아키텍처, (1)이 입력이미지와 사용자 프롬프트와 매칭되는 bounding box 탐지, (2)가 bounding box를 기반으로 pixel-level mask 생성. 모델이 무거워 실시간 처리에는 부적합

- 실시간 및 경량화 모델의 발전

- 모바일 및 엣지 디바이스에서의 실시간 세분화를 위해 경량화된 모델들이 개발
    - YOLOv12-seg (2025): YOLO 시리즈의 최신 버전으로 객체 검출과 세분화를 동시에 수행, 실시간 처리에 적합
    - [MobileSAM](#): Segment Anything Model(SAM)의 경량화 버전

# Computer Vision

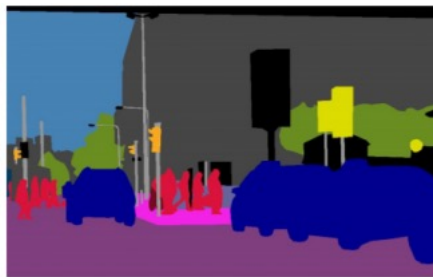
## Object Segmentation 최신 트렌드

### 트랜스포머 기반 세분화 모델의 부상

- 트랜스포머 아키텍처는 객체 세분화(segmentation) 분야에서도 주목받고 있음
- 세분화는 크게 semantic, instance, panoptic segmentation으로 나눌 수 있음
  - Semantic : 객체 인스턴스를 구별하지 않고, 클래스만 나눔
  - Instance segmentation: 객체 인스턴스 별 분리 (배경에는 관심 없음)
  - Panoptic segmentation: 배경은 semantic 로 분리, 객체는 instance로 분리



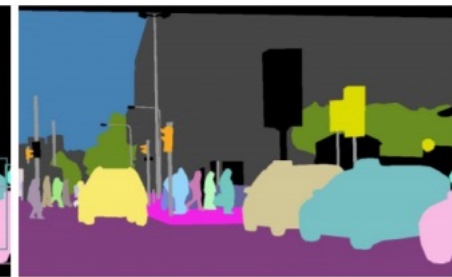
(a) image



(b) semantic segmentation



(c) instance segmentation



(d) panoptic segmentation

| 항목         | Semantic                     | Instance                                    | Panoptic                                           |
|------------|------------------------------|---------------------------------------------|----------------------------------------------------|
| 픽셀 클래스 분류  | ✓                            | ✓                                           | ✓                                                  |
| 객체 인스턴스 구분 | ✗                            | ✓                                           | ✓                                                  |
| "객체/배경" 구분 | ✗                            | ✗                                           | ✓                                                  |
| 대표 모델      | DeepLab (2016)<br>FCN (2014) | Mask R-CNN (2017)<br>Mask2Former (CVPR2022) | Panoptic FPN (CVPR 2019)<br>Mask2Former (CVPR2022) |

# Computer Vision

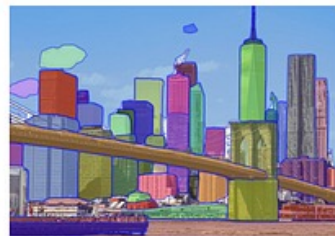
- Object Segmentation 최신 트렌드

- 멀티모달 및 프롬프트 기반 세분화

- [Segment Anything Model \(SAM\)](#) (2023): Meta에서 개발한 모델, 홈페이지에서 데모 제공, 다양한 프롬프트(텍스트 혹은 클릭한 지점, 박스)를 통해 세분화 수행
    - Grounded-SAM (2024): (1)Grounding DINO과 (2)SAM을 결합한 아키텍처, (1)이 입력이미지와 사용자 프롬프트와 매칭되는 bounding box 탐지, (2)가 bounding box를 기반으로 pixel-level mask 생성. 모델이 무거워 실시간 처리에는 부적합

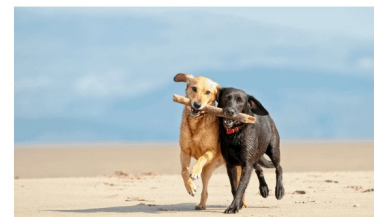


Prompt it with interactive points and boxes.

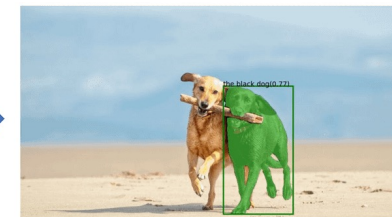


Automatically segment everything in an image.

SAM 결과



Text Prompt: The black dog



Text Prompt: the yellow chair



Grounded-SAM 입력과 결과

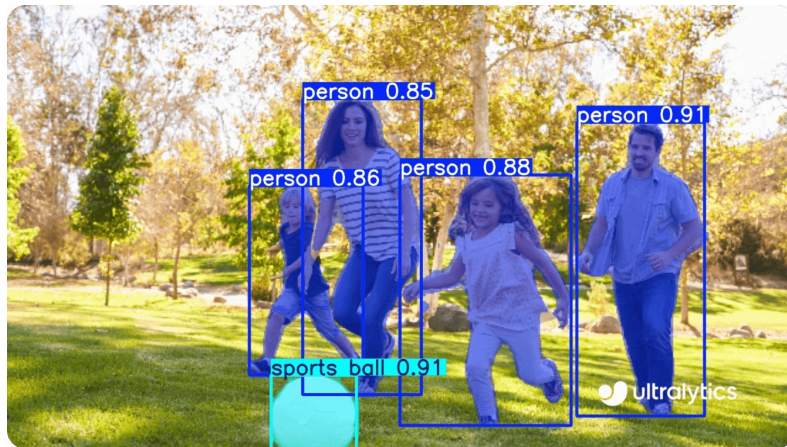


# Computer Vision

- Object Segmentation 최신 트렌드

- 실시간 및 경량화 모델의 발전

- 모바일 및 엣지 디바이스에서의 실시간 세분화를 위해 경량화된 모델들이 개발
    - YOLOv12-seg (2025): YOLO 시리즈의 최신 버전으로 객체 검출과 세분화를 동시에 수행, 실시간 처리에 적합
    - [MobileSAM](#): Segment Anything Model (SAM)의 경량화 버전



YOLOv11-instance seg



(a) Original SAM

(b) MobileSAM

(c) Original SAM

(d) MobileSAM

MobileSAM vs SAM with BB prompt  
(MobileSAM이 5배 작고, 7배 빠름)

# 정리

## ■ 시각 인지 기술의 큰 흐름

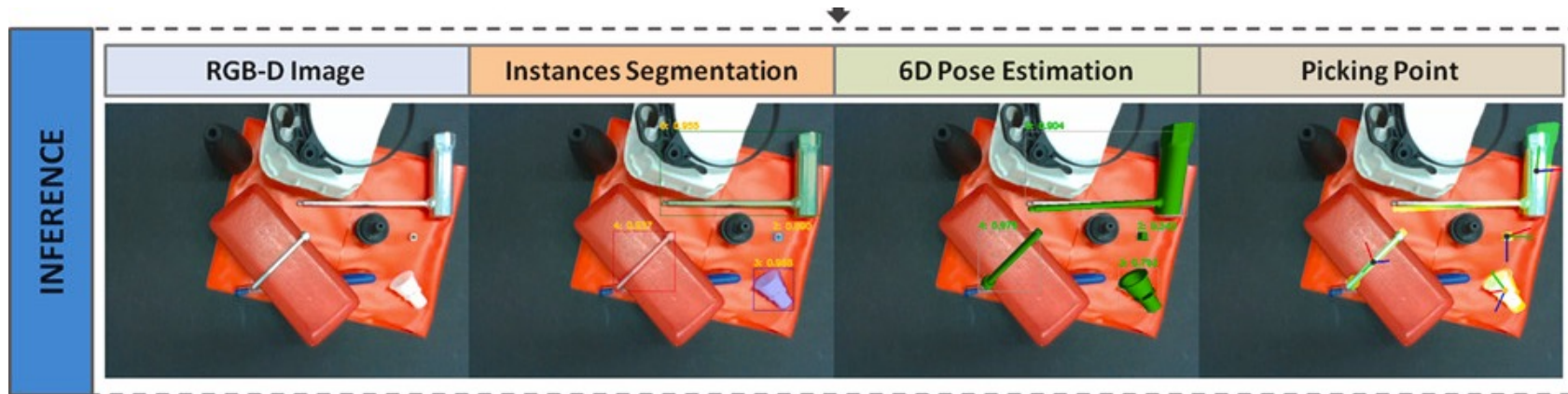
- CNN 아키텍처 기반 연구 (2012~)
  - 신경망 연구의 시작, 다양한 CNN 백본 모델을 제안하며 성능 향상을 가져옴
- Vision Transformer 아키텍처 기반 연구 (2021~)
  - 자연어 분야에서 발표된 Transformer (2017) 아키텍처를 비전 분야 아키텍처로 도입한 사례
  - 다양한 ViT 백본 모델을 제안하며 성능 향상을 가져옴
- **\*\* VLM (Vision Language Model) 기반 연구 (2021~)**
  - 기존 비전 모델은 수천~수만 개 클래스에 대해 고정된 Label만 탐지 가능 (Close-set)
  - VLM 모델은 텍스트 임베딩 기반으로 작동하기 때문에, 사전에 본 적 없는 클래스도 언어로 설명하면 탐지 가능 (Zero-shot 가능)

| Task                 | 전통적 방식               | VLM 기반 전환                           | 주요 기술                         |
|----------------------|----------------------|-------------------------------------|-------------------------------|
| Image Classification | ResNet, EfficientNet | CLIP                                | 텍스트 설명과 이미지 간 유사도 비교          |
| Object Detection     | Faster R-CNN, YOLO   | Grounding DINO, OWL-ViT, YOLO-World | 프롬프트 기반 탐지 (Zero-shot 가능)     |
| Object Segmentation  | Mask R-CNN, DeepLab  | Grounded-SAM, OWL-Seg               | 프롬프트 기반 마스크 생성 (Zero-shot 가능) |



# Computer Vision

- “시각 인지 기술”별 최신 트렌드
  - 6D Object Pose Estimation
  - Affordance Estimation



(b) *pick up* v.s. *open* the pot



(c) hold *electric drill* v.s. use *frying pan*



# Computer Vision

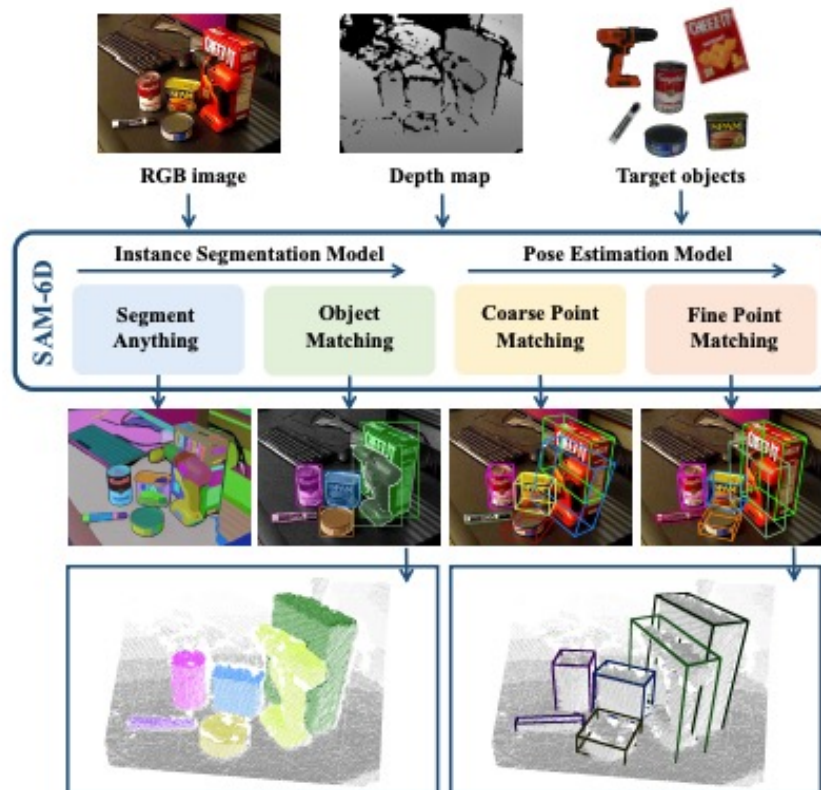
- 6D Object Pose Estimation + VLM 모델과의 만남

| 모델명            | 발표 연도               | 주요 특징                        | 제로샷<br>지원 여부 | Open-Vocabulary<br>지원 여부 |
|----------------|---------------------|------------------------------|--------------|--------------------------|
| Oryon          | CVPR 2024           | 텍스트 프롬프트 기반 6D 자세 추정         | O            | O                        |
| SAM-6D         | CVPR 2024           | SAM을 활용한 제로샷 객체 분할 및 자세 추정   | O            | O                        |
| ZS6D           | 2023                | ViT 기반 템플릿 매칭을 통한 자세 추정      | O            | X                        |
| ViTa-Zero      | 2025                | 시각 및 촉각 정보 결합을 통한 자세 추정      | O            | X                        |
| FoundationPose | CVPR 2024<br>NVIDIA | 사전 학습된 모델을 활용한 통합 자세 추정 및 추적 | O            | O                        |

# Computer Vision

- 6D Object Pose Estimation

- SAM-6D: Segment Anything Model Meets Zero-Shot 6D Object Pose Estimation (CVPR 2024)
  - Segment Anything Model (SAM)을 활용하여 RGB-D 이미지에서 제로샷으로 객체를 분할하고, 해당 객체의 6D 자세를 추정하는 프레임워크
    - CAD Model-Free / Template-Free 방법임



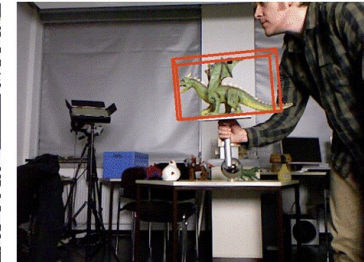
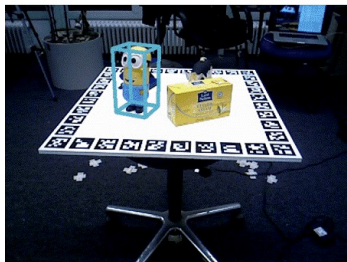
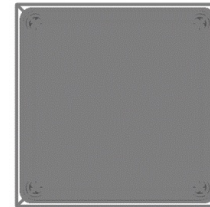
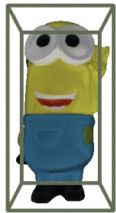
An overview of our proposed SAM-6D, which consists of an Instance Segmentation Model (ISM) and a Pose Estimation Model (PEM) for joint instance segmentation and pose estimation of novel objects in RGB-D images.

ISM leverages the Segment Anything Model (SAM) to generate all possible proposals and selectively retains valid ones based on object matching scores.

PEM involves two stages of point matching, from coarse to fine, to establish 3D-3D correspondence and calculate object poses for all valid proposals.

# Computer Vision

- 6D Object Pose Estimation + VLM 모델과의 만남
  - SAM-6D: Segment Anything Model Meets Zero-Shot 6D Object Pose Estimation
    - CVPR 2024
    - Segment Anything Model (SAM)을 활용하여 RGB-D 이미지에서 제로샷으로 객체를 분할하고, 해당 객체의 6D 자세를 추정하는 프레임워크





# Computer Vision

- Affordance Estimation + VLM 모델과의 만남

| 모델명           | 발표 연도         | 주요 특징                  | 제로샷<br>지원 여부 | Open-Vocabulary<br>지원 여부 |
|---------------|---------------|------------------------|--------------|--------------------------|
| AffordanceLLM | CVPRW<br>2024 | VLM 기반 어포던스 그라운드링      | O            | O                        |
| RoboPoint     | 2024          | 언어 지시 기반 키포인트 어포던스 예측  | O            | O                        |
| PAVLM         | 2024          | 포인트 클라우드 기반 3D 어포던스 추론 | O            | O                        |
| GLOVER        | 2024          | 오픈 어휘 기반 그립 어포던스 추론    | O            | O                        |
| INTRA         | 2024          | 약한 감독 하의 어포던스 그라운드링    | O            | O                        |

# Computer Vision

## ▪ Affordance Estimation + VLM 모델과의 만남

### ▪ GLOVER, 2024

- Open-Vocabulary 기반의 그리퍼 어포던스 추론을 위해,

대규모 언어 모델을 활용하여 RGB 이미지에서 객체의 그립 가능한 부위를 예측하는 프레임워크를 제안

- 입력데이터 : 영상 + 텍스트 (덱스 사용 안함)
- 10,000개 이상의 인간-객체 상호작용 이미지로 학습, 실시간 어포던스 추론 및 “그립 포즈” 예측 가능

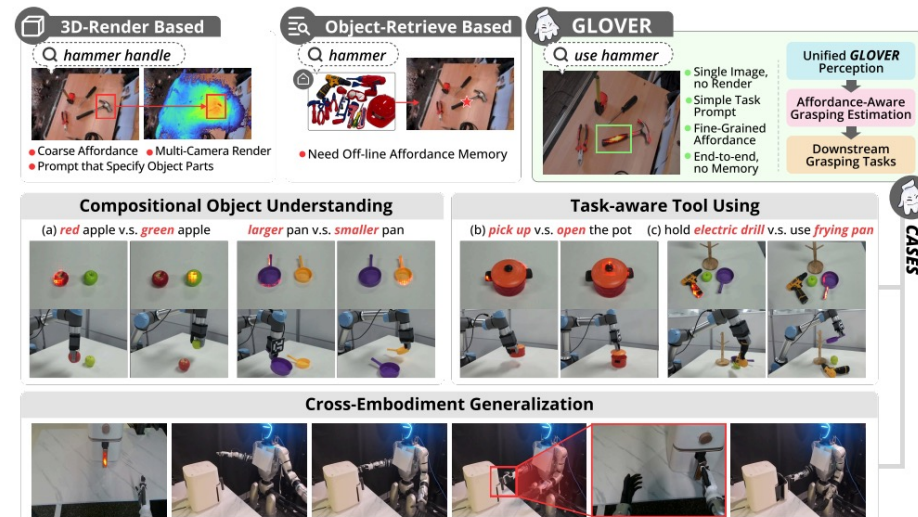


Figure 1. **Top:** Compared with previous methods, GLOVER eliminates the need for capturing multi-view images for 3D rendering, explicit instruction of the grasping part, and building extra off-line affordance memory. GLOVER is capable of providing more accurate fine-grained affordance predictions. **Mid:** We demonstrate the efficacy of GLOVER in handling complex scenarios involving **compositional object understanding** and **task-aware tool using** under targeted human instructions, including: (a) attributes and relations like color, size, material, (b) tool function reasoning according to action, (c) common tool using in complex scenes. **Bottom:** We validate the effectiveness across embodiments (in a humanoid robot with dexterous hands), the robot grasps the oven handle based on inferred affordance.



# Computer Vision

- Affordance Estimation + VLM 모델과의 만남
  - GLOVER, 2024
    - Open-Vocabulary 기반의 그립 어포던스 추론을 위해,  
대규모 언어 모델을 활용하여 RGB 이미지에서 객체의 그립 가능한 부위를 예측하는 프레임워크를 제안
    - 10,000개 이상의 인간-객체 상호작용 이미지로 학습, 실시간 어포던스 추론 및 그립 포즈 예측 가능

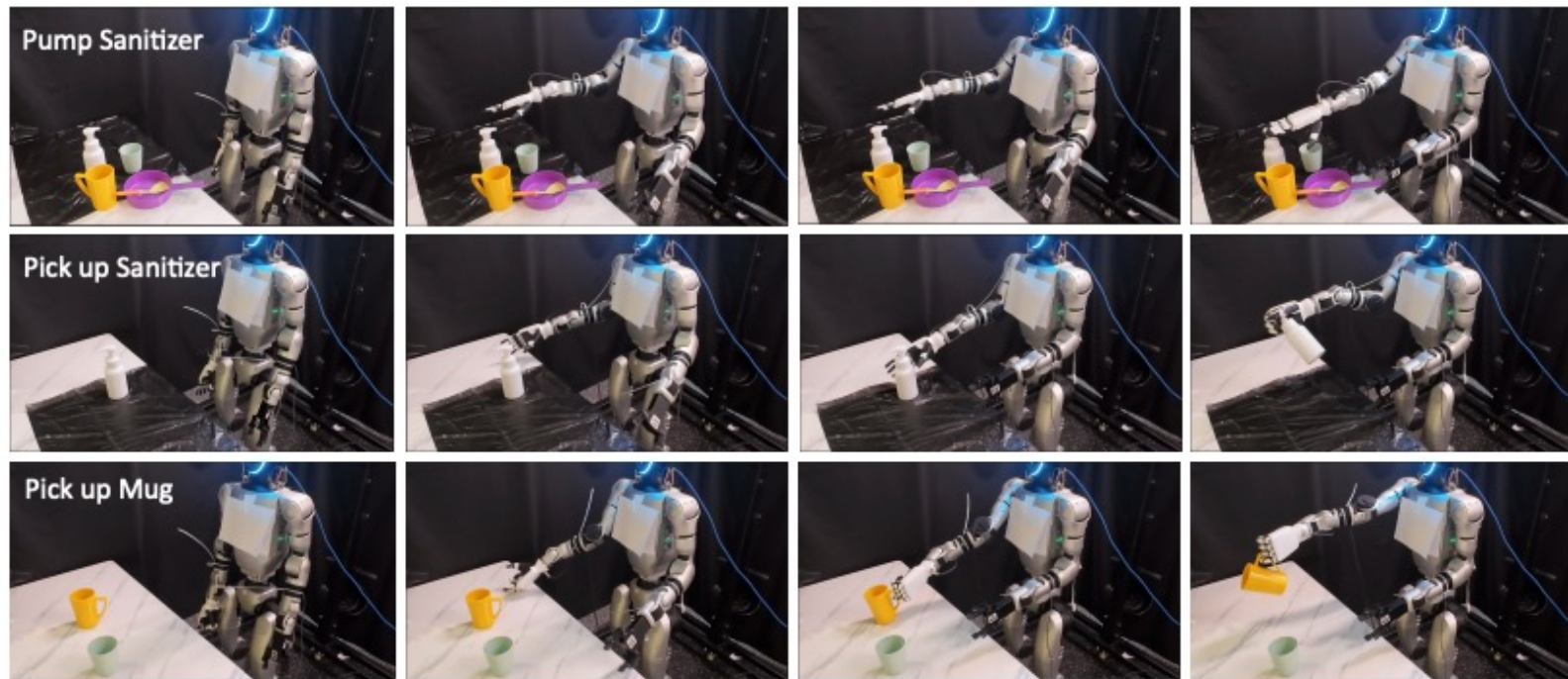


Figure 7. Demonstrations of manipulation tasks completed by the Unitree G1.